



RIGA TECHNICAL UNIVERSITY
FACULTY OF CIVIL AND MECHANICAL ENGINEERING
INSTITUTE OF MECHANICAL, AEROSPACE AND TRANSPORT ENGINEERING

Furkan Batur Tavli

Student of the
Academic Master (Second Cycle) Studies
Master Thesis
„Aviation Transport”

(Student ID No. 231AMM063)

EVALUATION OF CLASSICAL FAULT DETECTION METHODS FOR AIRCRAFT AIR DATA SENSORS USING SIMULATED FLIGHT DATA

Scientific Advisor	
Scientific degree, position	<u>Associate Professor – Lead Researcher, Ph. D.</u>
Name SURNAME	<u>Ali Arshad</u>



RIGA TECHNICAL UNIVERSITY
FACULTY OF CIVIL AND MECHANICAL
ENGINEERING
INSTITUTE OF MECHANICAL,
AEROSPACE AND TRANSPORT
ENGINEERING



Date: _____

Registration No.: _____

Study program: **Aviation transport**

CONFIRMED

RTU FCME Deputy Director for Academic Affairs
of the Aeronautics, Space Engineering and
Transport Institute
Dr. chem., ing. techn. **A. Rijkuris**

(signature)

202 ____ . _____

TASK OF THE ACADEMIC MASTER'S THESIS

Student	Furkan Batur Tavli (name, surname)
Topic of the Academic <i>Master's Thesis</i> :	Evaluation of Classical Fault Detection Methods for Aircraft Air Data Sensors Using Simulated Flight Data
Tasks:	
	1. Simulate representative aircraft airspeed profiles covering climb, cruise, and descent flight phases.
	2. Model constant bias (+10 kt) and frozen signal faults in the simulated airspeed data.
	3. Implement threshold-based detection and rate-of-change monitoring algorithms in MATLAB.
	4. Evaluate detection performance quantitatively in terms of detection delay, missed detection rate, and false alarm occurrences.
	5. Interpret the results from an avionics engineering perspective and identify the practical strengths and limitations of each method.

Scientific advisor:	Associate Professor – Lead Researcher, Ph. D. Ali Arshad (position, scientific degree, name, surname, signature, date)
Consultant:	 (position, scientific degree, name, surname, signature, date)
	<i>Submission date of the finished work</i> _____
Task received:	 (date, student's signature)
<i>The Task has to be filled in two copies: 1- for the head of the study direction; 2 – for the student</i>	

ABSTRACT

Air data systems provide essential flight parameters including indicated airspeed, altitude, and Mach number, which are fundamental to aircraft control, flight envelope protection, and operational safety. Despite the high reliability of modern air data sensors, faults such as constant bias and frozen signal may occur due to pitot tube blockage, contamination, icing, or internal malfunction. Timely detection of such faults is necessary to maintain system integrity and prevent the propagation of incorrect data to flight-critical systems.

The aim of this thesis is to evaluate the effectiveness of classical fault detection methods for detecting constant bias and frozen signal faults in aircraft airspeed data using simulated flight profiles. Threshold-based and rate-of-change detection algorithms are implemented in MATLAB and evaluated through a Monte Carlo campaign of 600 simulation runs across twelve scenario combinations of fault type, noise level, and flight phase. Performance is assessed using mean detection delay, missed detection rate, and mean false alarm event count per run.

Both methods detected all injected bias faults with negligible delay across all tested conditions. For frozen signal faults, the threshold-based method failed entirely under low and medium noise conditions with a missed detection rate of 100%, while the rate-of-change method achieved zero missed detections with a consistent mean detection delay of approximately 5 seconds. False alarm frequency increased substantially under high noise and was identical between both methods due to a shared detection mechanism. These findings confirm that neither method alone provides complete fault coverage and that layered monitoring architectures are necessary for comprehensive airspeed sensor fault detection.

Key words: air data system, airspeed sensor, fault detection, threshold monitoring, rate-of-change monitoring, frozen signal, constant bias, Monte Carlo simulation, avionics, MATLAB.

ANOTĀCIJA

Gaisa datu sistēmas nodrošina būtiskus lidojuma parametrus, tostarp rādīto gaisa ātrumu, augstumu un Mača skaitli, kas ir būtiski gaisa kuģa vadībai, lidojuma robežparametru ievērošanai un ekspluatācijas drošībai. Neskatoties uz mūsdienu gaisa datu sensoru augsto uzticamību, tādu kļūdu kā pastāvīga nobīde un signāla iesaldēšanās var rasties pītota caurules aizsērēšanas, piesārņojuma, apledošanas vai iekšējās darbības traucējumu dēļ. Šādu kļūdu savlaicīga atklāšana ir nepieciešama, lai saglabātu sistēmas integritāti un novērstu nepareizu datu nonākšanu lidojumam kritiskās sistēmās.

Šīs disertācijas mērķis ir novērtēt klasisko kļūdu noteikšanas metožu efektivitāti pastāvīgas nobīdes un iesaldēta signāla kļūdu noteikšanai gaisa kuģa gaisa ātruma datos, izmantojot simulētus lidojuma profilus. Robežvērtību un izmaiņu ātruma noteikšanas algoritmi ir īstenoti MATLAB vidē un novērtēti, izmantojot 600 simulāciju Monte Carlo kampaņu, kas aptver divpadsmit scenāriju kombinācijas, kurās iekļauti kļūdu veidi, trokšņa līmenis un lidojuma fāzes. Veiktspēja tiek novērtēta, izmantojot vidējo atklāšanas kavējumu, neuzklausīto atklājumu rādītāju un vidējo viltus trauksmes gadījumu skaitu vienā ciklā.

Abas metodes atklāja visas ievadītās nobīdes kļūdas ar niecīgu kavējumu visos testētajos apstākļos. Attiecībā uz iesaldēto signālu kļūdām sliekšņa metode pilnībā neizdevās zemas un vidējas trokšņa līmeņa apstākļos ar neuzklausīto atklājumu rādītāju 100 %, savukārt izmaiņu ātruma metode sasniedza nulles neuzklausīto atklājumu rādītāju ar nemainīgu vidējo atklāšanas kavējumu aptuveni 5 sekundes. Viltus trauksmes biežums ievērojami palielinājās augsta trokšņa apstākļos un abām metodēm bija identisks, jo tām bija kopīgs atklāšanas mehānisms. Šie rezultāti apstiprina, ka neviena no metodēm atsevišķi nenodrošina pilnīgu kļūdu pārklājumu un ka visaptverošai gaisa ātruma sensora kļūdu atklāšanai ir nepieciešamas daudzslāņu uzraudzības arhitektūras.

Atslēgvārdi: gaisa datu sistēma, gaisa ātruma sensors, kļūdu atklāšana, sliekšņa uzraudzība, izmaiņu ātruma uzraudzība, iesaldēts signāls, konstanta nobīde, Monte Carlo simulācija, avi.

ÖZET

Hava veri sistemleri, uçak kontrolü, uçuş sınırları koruması ve operasyonel güvenlik açısından temel öneme sahip olan hava hızı, irtifa ve Mach sayısı gibi hayati uçuş parametrelerini sağlar. Modern hava veri sensörlerinin yüksek güvenilirliğine rağmen, pitot tüpünün tıkanması, kirlenme, buzlanma veya dahili arızalar nedeniyle sabit sapma ve donmuş sinyal gibi arızalar meydana gelebilir. Sistem bütünlüğünü korumak ve hatalı verilerin uçuş açısından kritik sistemlere yayılmasını önlemek için bu tür arızaların zamanında tespit edilmesi gereklidir.

Bu tezin amacı, simüle edilmiş uçuş profilleri kullanarak uçak hava hızı verilerindeki sabit sapma ve donmuş sinyal arızalarını tespit etmek için klasik arıza tespit yöntemlerinin etkinliğini değerlendirmektir. Eşik tabanlı ve değişim oranı tespit algoritmaları MATLAB'da uygulanmış ve arıza türü, gürültü seviyesi ve uçuş aşamasının on iki senaryo kombinasyonunda yürütülen 600 simülasyondan oluşan bir Monte Carlo simulasyonu aracılığıyla değerlendirilmiştir. Performans, ortalama algılama gecikmesi, kaçırılan algılama oranı ve her çalıştırma başına ortalama yanlış alarm sayısı kullanılarak değerlendirilmiştir.

Her iki yöntem de, test edilen tüm koşullarda ihmal edilebilir bir gecikmeyle enjekte edilen tüm sapma arızalarını tespit etmiştir. Donmuş sinyal arızaları için, eşik tabanlı yöntem düşük ve orta gürültü koşullarında %100 kaçırılan algılama oranıyla tamamen başarısız olurken, değişim oranı yöntemi yaklaşık 5 saniyelik tutarlı bir ortalama algılama gecikmesi ile sıfır kaçırılan algılama elde etmiştir. Yanlış alarm sıklığı, yüksek gürültü altında önemli ölçüde artmıştır ve ortak bir algılama mekanizması nedeniyle her iki yöntem arasında aynıdır. Bu bulgular, hiçbir yöntemin tek başına tam arıza kapsamı sağlamadığını ve kapsamlı hava hızı sensörü arıza algılaması için katmanlı izleme mimarilerinin gerekli olduğunu doğrulamaktadır.

Anahtar kelimeler: hava veri sistemi, hava hızı sensörü, arıza algılama, eşik izleme, değişim oranı izleme, donmuş sinyal, sabit sapma, Monte Carlo simülasyonu, aviyonik, MATLAB.

TABLE OF CONTENTS

ABSTRACT	4
ANOTĂCIJA	5
ÖZET	6
ABBREVIATIONS.....	9
INTRODUCTION	10
1. LITERATURE REVIEW	12
1.1. AIR DATA SYSTEMS IN AIRCRAFT.....	12
1.1.1. Architecture of the Pitot-Static System	13
1.1.2. Air Data Computer Processing and Signal Conditioning	14
1.1.3. Operational Importance of Airspeed in Aircraft Systems	16
1.2. AIRSPEED MEASUREMENT PRINCIPLES	17
1.3. COMMON AIR DATA SENSOR FAILURE MODES.....	19
1.3.1. Pitot Tube Blockage	19
1.3.2. Static Port Blockage	20
1.3.3. Constant Bias and Calibration Errors	20
1.3.4. Electronic and Signal Processing Disturbances.....	21
1.3.5. Justification of Selected Fault Types.....	21
1.3.6. Historical Aviation Incidents Involving Air Data Sensor Faults	22
1.4. CLASSICAL FAULT DETECTION METHODS IN AVIONICS	24
1.4.1. Threshold-Based Monitoring.....	24
1.4.2. Rate-of-Change Monitoring	25
1.4.3. Residual-Based Monitoring.....	27
1.4.4. Redundancy and Cross-Comparison Logic	28
1.4.5. Rationale for Selected Detection Methods	29
1.5. PERFORMANCE METRICS IN FAULT DETECTION.....	30
1.5.1. Detection Delay	30
1.5.2. Missed Detection	30
1.5.3. False Alarm	31
1.5.4. Trade-Off Between Sensitivity and Robustness	31
1.5.5. Evaluation Approach in This Thesis	32
1.6. SUMMARY OF LITERATURE AND RESEARCH GAP.....	32
1.7. REGULATORY AND CERTIFICATION CONTEXT FOR AIR DATA MONITORING	33
2. METHODOLOGY	36
2.1. METHODOLOGICAL FRAMEWORK	36
2.2. AIRSPEED SIGNAL SIMULATION.....	37
2.2.1. Modelling Assumptions.....	37
2.2.2. Flight Profile Structure	37
2.2.3. Piecewise Airspeed Profile Definition	38
2.3. NOISE MODELING	38
2.4. FAULT MODELING	40
2.4.1. Constant Bias Fault	40
2.4.2. Frozen Signal Fault	41
2.5. FAULT DETECTION IMPLEMENTATION	41
2.5.1. Threshold-Based Detection	42
2.5.2. Rate-of-Change Monitoring	42
2.6. EXPERIMENTAL DESIGN.....	43
2.6.1. Scenario Matrix	44

2.6.2.	Fault Onset Time	44
2.6.3.	Number of Simulation Runs	44
2.7.	PERFORMANCE EVALUATION PROCEDURE.....	45
2.7.1.	Detection Delay Calculation.....	45
2.7.2.	Missed Detection Rate.....	45
2.7.3.	False Alarm Rate	45
2.8.	STATISTICAL POST-PROCESSING.....	46
3.	RESULTS	47
3.1.	OVERVIEW OF SIMULATION CAMPAIGN.....	47
3.2.	DETECTION PERFORMANCE FOR BIAS FAULTS.....	47
3.3.	DETECTION PERFORMANCE FOR FREEZE FAULTS	48
3.4.	FALSE ALARM ANALYSIS	48
3.5.	OVERALL SUMMARY OF RESULTS	49
3.6.	VISUALIZATION OF DETECTION PERFORMANCE.....	50
4.	DISCUSSION	55
4.1.	INTERPRETATION OF BIAS FAULT DETECTION RESULTS	55
4.2.	INTERPRETATION OF FREEZE FAULT DETECTION RESULTS.....	56
4.3.	FALSE ALARM BEHAVIOR AND PHASE DEPENDENCY	59
4.4.	COMPARATIVE ASSESSMENT OF DETECTION METHODS	61
4.5.	LIMITATIONS OF THE STUDY	62
4.6.	PRACTICAL IMPLICATIONS AND ENGINEERING RECOMMENDATIONS	63
4.7.	DIRECTIONS FOR FUTURE RESEARCH.....	65
	CONCLUSIONS.....	67
	BIBLIOGRAPHY	70
	APPENDICES.....	72
	APPENDIX NO. 1.....	73
	APPENDIX NO. 2.....	75
	APPENDIX NO. 3.....	77
	APPENDIX NO. 4.....	78
	APPENDIX NO. 5.....	79
	APPENDIX NO. 6.....	81
	APPENDIX NO. 7.....	83
	APPENDIX NO. 8.....	86
	APPENDIX NO. 9.....	88
	APPENDIX NO. 10.....	92
	APPENDIX NO. 11.....	100
	APPENDIX NO. 12.....	112

ABBREVIATIONS

Abbreviation	Full Term / Definition	Unit / Note
ADC	Air Data Computer	—
ADIRU	Air Data Inertial Reference Unit	—
BEA	Bureau d'Enquêtes et d'Analyses pour la Sécurité de l'Aviation Civile	—
BIT	Built-In Test	—
CAS	Calibrated Airspeed	kt
CS-25	Certification Specifications for Large Aeroplanes (EASA)	—
DO-160	Environmental Conditions and Test Procedures for Airborne Equipment (RTCA)	—
EASA	European Union Aviation Safety Agency	—
FAA	Federal Aviation Administration	—
FDI	Fault Detection and Isolation	—
FMS	Flight Management System	—
Hz	Hertz (unit of sampling frequency)	s ⁻¹
IAS	Indicated Airspeed	kt
kt	Knot (unit of airspeed, 1 kt = 0.5144 m/s)	m/s
MATLAB	Matrix Laboratory (numerical computing environment)	—
PFD	Primary Flight Display	—
RTCA	Radio Technical Commission for Aeronautics	—
RTU	Riga Technical University	—
s	Second (unit of time)	—
TAS	True Airspeed	kt

INTRODUCTION

Modern aircraft rely on air data systems to provide critical flight parameters including airspeed, altitude, and Mach number. Among these, airspeed is one of the most fundamental variables used by flight control systems, autopilot functions, flight envelope protection, and pilot situational awareness. Reliable operation of air data sensors is therefore essential for safe aircraft operation across all phases of flight.

Despite their importance, air data sensors may experience faults caused by pitot tube blockage, contamination, icing, sensor degradation, or internal signal processing failures. Two representative fault types encountered in operational environments are constant bias, in which the sensor output is shifted by a fixed offset while preserving its dynamic characteristics, and frozen signal, in which the sensor output stops updating while the true flight parameter continues to change. Historical accident investigations have demonstrated that undetected or ambiguous airspeed anomalies can contribute to catastrophic outcomes even in aircraft equipped with redundant air data channels, highlighting the operational significance of robust fault detection.

In certified avionics systems, classical fault detection methods are commonly employed due to their simplicity, transparency, and suitability for formal verification under applicable airworthiness standards. These methods typically include threshold-based monitoring and rate-of-change analysis. Although widely implemented, their quantitative detection performance under varying flight conditions and noise levels is not always explicitly characterized in a structured manner.

The object of this research is the aircraft airspeed sensor signal within a single-channel air data system. The aim of the thesis is to evaluate the effectiveness of classical fault detection methods in detecting constant bias and frozen signal faults using simulated flight data.

To achieve this aim, the following tasks are defined:

- a) To review the technical background of air data systems, common airspeed sensor failure modes, classical fault detection methods, performance metrics, and the relevant regulatory context.
- b) To simulate representative airspeed profiles for climb, cruise, and descent phases with additive Gaussian measurement noise at multiple levels.

- c) To model and inject constant bias and frozen signal faults at randomized onset times within dynamic flight phases.
- d) To implement threshold-based and rate-of-change detection algorithms in MATLAB and execute a Monte Carlo simulation campaign across a full factorial experimental design.
- e) To evaluate detection performance in terms of mean detection delay, missed detection rate, and false alarm occurrence, with statistical post-processing performed in Python.
- f) To interpret the results from an avionics engineering perspective and derive practical engineering recommendations.

The scope is limited to a single sensor channel, two fault types, and two classical detection methods. Advanced techniques such as model-based observers, machine learning, and multi-sensor fusion are outside the scope of this work. The analysis is performed offline using simulated data to ensure controlled and reproducible evaluation conditions.

The expected outcome is a structured and quantitative comparison of classical fault detection methods that clearly identifies the conditions under which each method succeeds or fails, providing a practical basis for informed monitoring system design decisions.

1. LITERATURE REVIEW

The purpose of this chapter is to provide the theoretical and technical background necessary for understanding air data systems and classical fault detection methods applied to airspeed measurements. The first chapter presents the structure and operation of aircraft air data systems, with emphasis on airspeed sensing and processing. Common failure modes affecting air data sensors are then reviewed to justify the selection of representative fault types considered in this thesis. Subsequently, classical fault detection principles used in avionics systems are discussed, followed by an overview of commonly used performance evaluation metrics in fault detection analysis.

By establishing this background, the chapter defines the technical context of the research problem and identifies the need for structured evaluation of classical fault detection techniques under controlled simulation conditions.

1.1. Air Data Systems in Aircraft

The air data system is a fundamental subsystem of modern aircraft avionics architecture, responsible for measuring and providing essential flight parameters derived from atmospheric pressure measurements. These parameters typically include indicated airspeed (IAS), calibrated airspeed (CAS), Mach number, altitude, and vertical speed. The accuracy and reliability of these measurements are critical for safe aircraft operation, as they directly influence flight control systems, autopilot functions, flight envelope protection, and pilot situational awareness [1], [2].

In conventional configurations, the air data system consists of external pressure sensing elements—primarily the pitot tube and static ports—connected to an Air Data Computer (ADC). The pitot tube measures total (stagnation) pressure, while static ports measure ambient static pressure. The ADC processes these pressure measurements using established physical relationships to compute airspeed and altitude-related parameters [2]. In modern aircraft, air data functions may be integrated within Air Data Inertial Reference Units (ADIRUs), combining air data measurements with inertial sensor information for redundancy and system robustness [3].

Indicated airspeed (IAS) is one of the most operationally significant outputs of the air data system. It represents the airspeed value directly presented to pilots and used by various onboard systems. IAS plays a crucial role in takeoff performance, climb scheduling, cruise

efficiency, stall margin protection, and approach stability. Many flight control laws and protection mechanisms rely on airspeed thresholds, making the integrity of this signal essential for safe flight operations [7], [8].

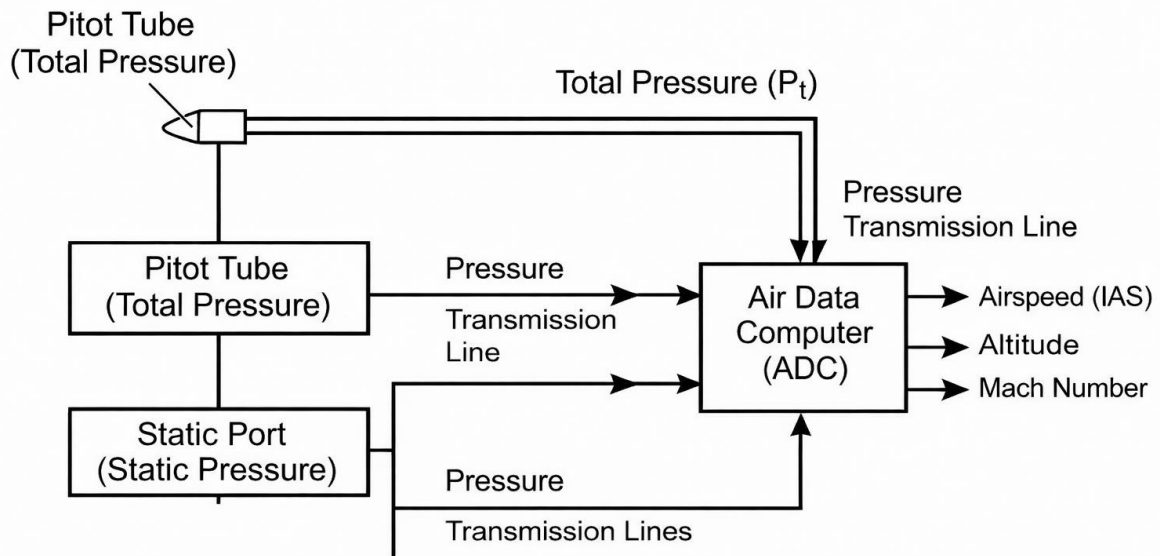
Although air data systems are designed with redundancy and reliability in mind, they remain susceptible to measurement disturbances and sensor faults. Environmental factors such as icing, contamination, and blockage of pressure lines can affect the accuracy of pressure measurements. Additionally, internal sensor degradation or data processing errors may introduce systematic deviations in computed airspeed values [2], [11]. For this reason, avionics architectures typically incorporate monitoring logic to detect abnormal sensor behavior and prevent the propagation of incorrect air data throughout the aircraft systems.

In commercial aviation operations, reliable airspeed indication is particularly important due to high utilization rates, varying environmental conditions, and the need for strict adherence to operational limits. Maintenance organizations and airline technical departments routinely address air data-related discrepancies, underscoring the operational relevance of fault detection mechanisms within this subsystem. However, despite built-in redundancies and monitoring features, understanding the behavior of classical fault detection approaches under controlled conditions remains valuable for engineering analysis and system evaluation.

In summary, the air data system constitutes a critical interface between the aircraft and its surrounding atmosphere, converting pressure measurements into actionable flight information. Given the central role of airspeed in flight control and monitoring functions, evaluating the effectiveness of fault detection methods applied to airspeed sensor data represents a meaningful and practically relevant research objective.

1.1.1. Architecture of the Pitot-Static System

The pitot-static system forms the sensing backbone of the air data system. A simplified representation of the pitot-static architecture is shown in Picture 1.1. Its primary purpose is to measure atmospheric pressures from which essential flight parameters are derived. The system typically consists of a pitot tube, one or more static ports, pressure transmission lines, and an Air Data Computer (ADC) [1], [2].



Pic. 1.1. Simplified architecture of a typical pitot-static air data system. [1]

The pitot tube is mounted externally on the aircraft fuselage or wing and is oriented into the airflow. It measures total (stagnation) pressure, which represents the sum of static pressure and dynamic pressure resulting from the aircraft's forward motion. Static ports, positioned flush with the fuselage surface, measure ambient atmospheric pressure. The difference between total and static pressure corresponds to dynamic pressure, from which airspeed can be derived.

In transport-category aircraft, redundancy is a fundamental design principle. Typically, multiple independent pitot-static systems are installed, supplying data to separate ADC channels [2], [3]. For example, captain-side, first-officer-side, and standby systems may each have independent pressure sources and processing units [7]. This redundancy enhances system reliability and allows cross-comparison logic within avionics architectures.

Pressure transmission lines connect the external sensors to the ADC. These lines must maintain integrity to prevent pressure leaks or contamination. Blockages, moisture accumulation, or icing within these lines may alter pressure readings, resulting in abnormal sensor behavior. From an operational perspective, such disturbances are among the most common air data system anomalies encountered in service environments.

1.1.2. Air Data Computer Processing and Signal Conditioning

The Air Data Computer (ADC) converts measured pressure values into usable flight parameters using well-established aerodynamic and thermodynamic relationships. The primary

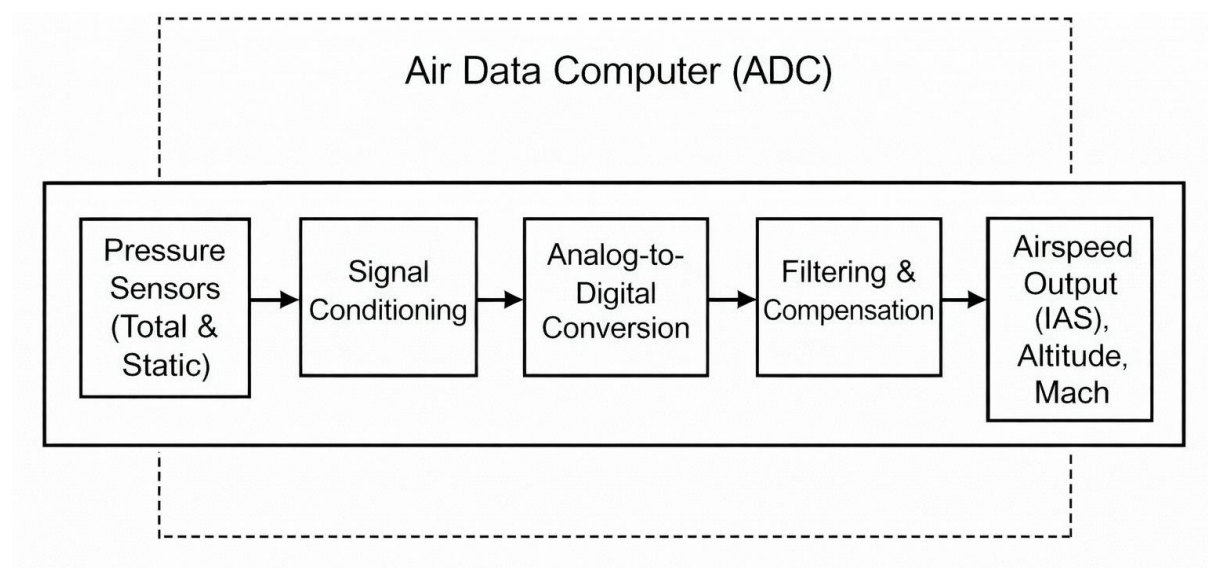
calculation involves determining indicated airspeed (IAS) from measured total and static pressures [1], [12].

In modern digital implementations, pressure sensors provide electrical signals proportional to pressure magnitude. These signals are digitized and processed by embedded microprocessors within the ADC. The processing chain typically includes:

- Signal filtering to reduce noise measurement.
- Compensation for temperature effects.
- Calibration corrections.
- Consistency checks between redundant channels.

Signal filtering is particularly relevant from a fault detection perspective. Excessive filtering may mask rapid changes, while insufficient filtering may increase susceptibility to noise-induced false alarms. Therefore, air data signal processing balances responsiveness with stability.

In addition to calculating airspeed, the ADC may perform internal monitoring functions. These include plausibility checks, channel comparison logic, and internal built-in test (BIT) routines [3], [9]. A simplified representation of the ADC processing chain is shown in Picture 1.2. While the present thesis focuses only on simplified external monitoring methods, it is important to recognize that real avionics systems implement multiple layers of validation to ensure data integrity.



Pic. 1.2. Simplified signal processing chain within a digital Air Data Computer. [1]

The processed airspeed signal is distributed to several aircraft subsystems, including:

- Primary Flight Display (PFD)
- Autopilot system
- Flight Management System (FMS)
- Flight control computers

This interconnected architecture highlights the importance of reliable airspeed data and justifies the need for effective fault detection mechanisms.

1.1.3. Operational Importance of Airspeed in Aircraft Systems

Airspeed is one of the most operationally sensitive flight parameters. Unlike altitude or heading, airspeed directly influences aerodynamic forces and aircraft performance margins. Several critical functions depend explicitly on accurate airspeed measurements [7], [8].

During takeoff and climb, airspeed is used to determine rotation speed, climb scheduling, and safe acceleration. During cruise, it contributes to performance optimization and fuel efficiency management. In descent and approach phases, airspeed control is essential for maintaining stable approach criteria and stall margin.

Modern fly-by-wire aircraft incorporate flight envelope protection logic, which relies heavily on airspeed thresholds. Stall protection systems monitor low-speed conditions, while overspeed protection prevents structural exceedance. Autopilot modes such as speed hold or managed speed also depend on accurate airspeed input.

From a maintenance and operational perspective, erroneous airspeed indications may lead to dispatch delays, maintenance troubleshooting actions, or operational restrictions. Air data discrepancies are routinely investigated in airline technical departments, further emphasizing the practical importance of reliable fault detection.

Given the central role of airspeed in flight safety and aircraft system integration, evaluating the effectiveness of fault detection methods applied to airspeed data is both technically meaningful and operationally relevant.

1.2. Airspeed Measurement Principles

Airspeed represents the velocity of an aircraft relative to the surrounding air mass and is one of the most fundamental aerodynamic parameters in flight mechanics. From a measurement perspective, airspeed is not directly sensed as velocity; instead, it is derived from pressure measurements obtained through the pitot-static system. The relationship between pressure and airspeed is based on Bernoulli's principle, where the dynamic pressure is proportional to the square of the aircraft's velocity relative to the airflow [12].

The total pressure p_t measured by the pitot tube consists of static pressure p_s and dynamic pressure q , such as:

$$p_t = p_s + q \quad (1.1)$$

where: p_s – static pressure

q – dynamic pressure

The dynamic pressure q can be expressed as:

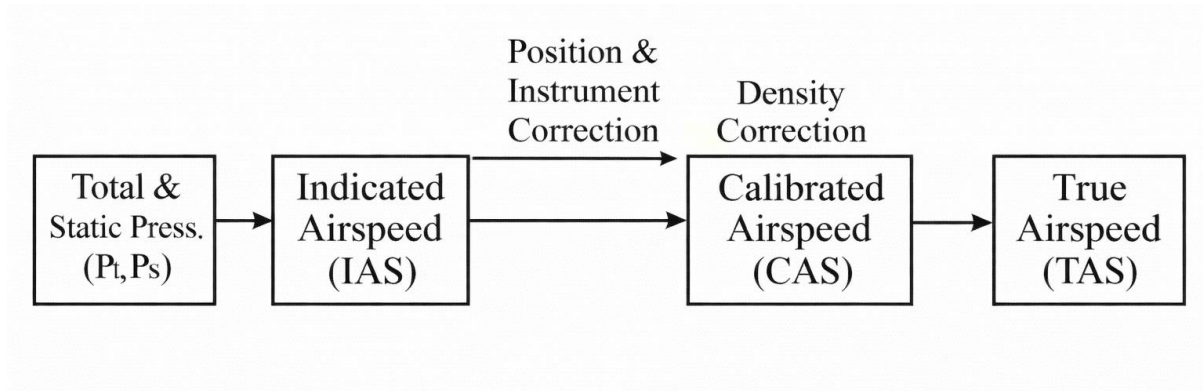
$$q = \frac{1}{2} \rho V^2 \quad (1.2)$$

where: ρ – is air density

V – is the true airspeed

In practical avionics systems, the Air Data Computer processes measured pressure values to compute indicated airspeed (IAS), which is the airspeed value displayed to pilots and used by onboard systems.

Indicated airspeed differs from calibrated airspeed (CAS) and true airspeed (TAS) due to instrument and installation errors. The conceptual relationship between these airspeed definitions is illustrated in Picture 1.3. IAS represents the raw measurement corrected for instrument characteristics but not fully corrected for position errors or compressibility effects. CAS includes corrections for position and calibration errors, while TAS further accounts for atmospheric density variations [7], [8].



Pic. 1.3. Conceptual relationship between IAS, CAS, and TAS in airspeed measurement. [1]

In operational contexts, IAS is the most relevant parameter for flight control and monitoring. Stall speeds, maneuvering speeds, and structural limitations are typically defined in terms of indicated or calibrated airspeed. Therefore, the integrity of IAS is particularly important for maintaining aerodynamic safety margins.

Measurement uncertainties in airspeed can arise from multiple sources. These include:

- Pressure sensor calibration deviations.
- Static port position errors.
- Blockage or partial obstruction of pressure lines.
- Environmental effects such as icing or contamination.
- Electronic noise within signal processing circuitry.

Such disturbances may introduce systematic deviations or random fluctuations in the airspeed signal. A systematic deviation may manifest as a constant bias, where the measured airspeed remains consistently offset from the true value. In contrast, dynamic disturbances may appear like increased noise or temporary anomalies.

From a signal analysis perspective, airspeed measurements exhibit characteristic behavior depending on the flight phase. During steady cruise, airspeed tends to remain relatively stable with small fluctuations. During climb or descent, moderate and smooth changes may occur as thrust and aerodynamic forces adjust. Abrupt large changes in airspeed over very short time intervals are generally unrealistic for transport-category aircraft and may indicate sensor malfunction or abnormal system behavior [12].

Understanding the normal dynamic behavior of airspeed signals is essential for designing and evaluating fault detection logic. Threshold-based monitoring relies on defining physically reasonable limits for airspeed values, while rate-of-change monitoring depends on expected temporal variations in the signal. Therefore, a clear understanding of airspeed measurement principles and expected signal characteristics forms the foundation for the subsequent fault modeling and detection analysis presented in this thesis.

1.3. Common Air Data Sensor Failure Modes

Air data systems are designed with redundancy and robustness; however, they remain vulnerable to a range of failure modes affecting pressure sensing and signal processing. Understanding these failure mechanisms is essential for selecting representative fault cases for evaluation.

Air data sensor failures can generally be categorized into environmental disturbances, mechanical blockages, calibration deviations, and internal electronic faults [6], [11]. Although modern systems include redundancy and built-in-test functions, certain fault types may still manifest as incorrect or abnormal airspeed indications.

1.3.1. Pitot Tube Blockage

One of the most widely recognized air data system faults involves blockage of the pitot tube. Blockage may occur due to icing, contamination, insect ingress, or debris accumulation. When the pitot tube becomes fully blocked while static pressure continues to vary, the measured total pressure may remain constant. As a result, the computed airspeed may exhibit a “frozen” behavior, remaining at a constant value despite changes in actual aircraft velocity [4], [11], [13].

If both pitot and static ports are blocked under certain conditions, the behavior may vary depending on the blockage configuration. However, in simplified analysis, pitot blockage often leads to airspeed signals that no longer respond dynamically to changes in flight conditions.

From a signal perspective, a frozen airspeed fault is characterized by:

- Absence of normal dynamic variation,
- Prolonged constant output despite changing flight phase,

- Unrealistically low rate of change.

Such characteristics justify the inclusion of a frozen signal fault in this thesis.

Historically, several aviation incidents have involved airspeed anomalies resulting from pitot tube obstruction or icing effects. These cases highlight the operational importance of reliable airspeed monitoring and fault detection. While modern aircraft include multiple air data sources and cross-check logic, simplified evaluation of frozen-signal detection remains academically relevant [6].

1.3.2. Static Port Blockage

Static port blockage may also affect airspeed computation. Since airspeed is derived from the difference between total and static pressure, incorrect static pressure measurements influence both altitude and airspeed indications.

Static port blockage can cause airspeed indications to deviate depending on flight conditions. In some cases, apparent airspeed may increase or decrease abnormally during climb or descent due to incorrect static pressure reference [2].

Although static blockage can produce more complex behavior than pure pitot blockage, its signal manifestation may resemble systematic deviation or bias under certain steady conditions.

1.3.3. Constant Bias and Calibration Errors

Systematic bias in airspeed measurement may arise from sensor calibration errors, pressure sensor drift, installation effects, or internal electronic deviations. Unlike abrupt faults, bias faults may not immediately violate absolute limits. Instead, the signal remains dynamically consistent but consistently offset from the true value.

From a signal analysis standpoint, constant bias faults exhibit:

- Normal dynamic behavior,
- Correct response to flight phase transitions,
- Persistent offset relative to expected values.

Such faults are particularly challenging for simple threshold-based detection methods because the measured value may remain within acceptable operational ranges while still being incorrect.

In operational contexts, small but persistent airspeed discrepancies are sometimes observed and may require maintenance troubleshooting or recalibration. Therefore, constant bias represents a realistic and meaningful fault case for evaluation [6], [11].

1.3.4. Electronic and Signal Processing Disturbances

In addition to mechanical blockages, internal electronic faults or data transmission errors may affect airspeed signals. These may include:

- Temporary signal dropouts,
- Excessive noise measurement,
- Quantization errors,
- Internal computational faults.

While advanced avionics architectures incorporate built-in-test functions to detect such anomalies, simplified external monitoring logic may still be necessary to identify abnormal signal behavior.

For the scope of this thesis, electronic disturbances are not modeled explicitly. However, understanding their existence reinforces the importance of robust monitoring mechanisms within air data systems.

1.3.5. Justification of Selected Fault Types

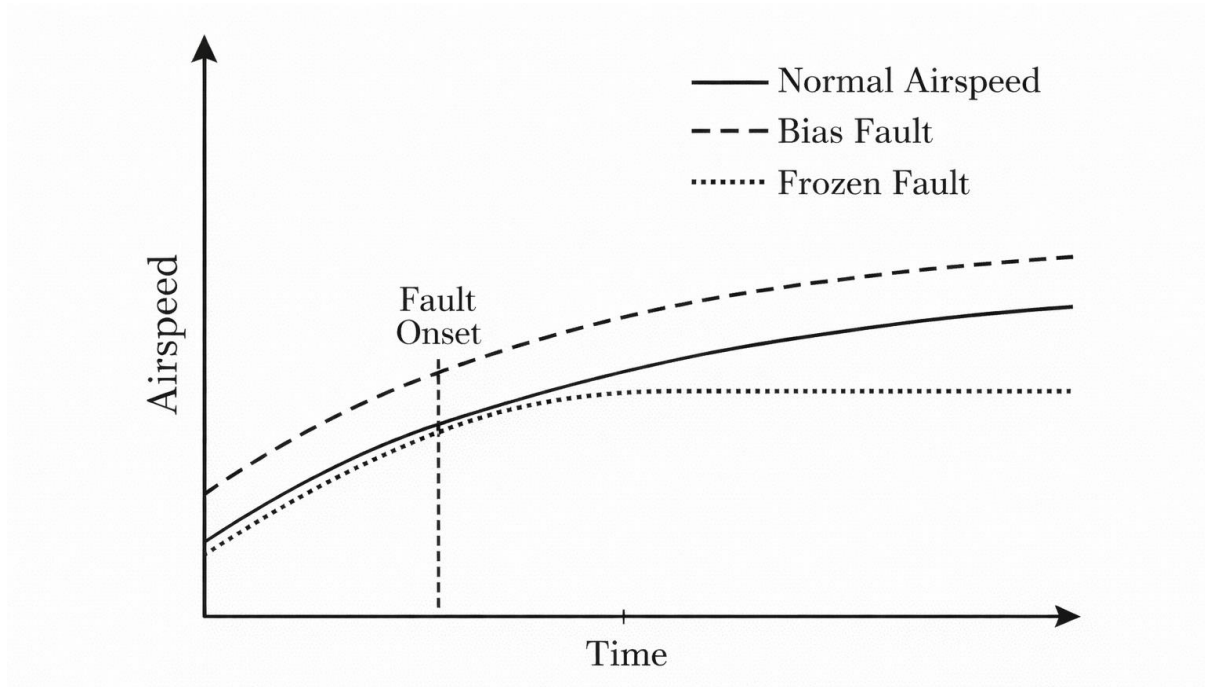
Based on the reviewed failure mechanisms, two representative fault types are selected for evaluation in this thesis:

- I.** Constant bias fault
- II.** Frozen (stuck) signal fault

These faults represent two fundamentally different classes of abnormal behavior:

- Bias faults preserve dynamic characteristics but alter absolute accuracy.
- Frozen faults disrupt dynamic response entirely.

Together, they provide a balanced basis for evaluating classical detection methods under controlled simulation conditions. The conceptual signal characteristics of these fault types are illustrated in Picture 1.4.



Pic. 1.4. Conceptual representation of bias and frozen airspeed faults. [1], [5]

1.3.6. Historical Aviation Incidents Involving Air Data Sensor Faults

The practical consequences of air data sensor failures are well documented in aviation accident and incident investigation records. Reviewing selected historical cases provides important operational context for the fault types modeled in this thesis and reinforces the engineering motivation for reliable fault detection mechanisms.

One of the most extensively analyzed events involving air data sensor failure is the loss of Air France Flight AF447 on 1 June 2009. The aircraft, an Airbus A330-203 operating between Rio de Janeiro and Paris, was lost over the Atlantic Ocean with 228 fatalities. The investigation conducted by the Bureau d'Enquêtes et d'Analyses pour la Sécurité de l'Aviation Civile (BEA) determined that the accident sequence was initiated by the temporary obstruction of the pitot probes due to ice crystal icing at high altitude. The obstruction caused the autopilot to disconnect and produced inconsistent airspeed indications across the redundant air data channels. The resulting crew confusion in response to conflicting airspeed data and the loss of reliable flight envelope protection contributed directly to the loss of control [13].

From a signal analysis perspective, the AF447 event illustrates how even brief and transient airspeed anomalies, when occurring simultaneously across multiple channels and without adequate crew alerting, can lead to catastrophic outcomes. The fault behavior observed in that event is consistent with characteristics of a frozen or unreliable airspeed signal during a dynamic flight phase — precisely the scenario modeled in this thesis through the frozen signal fault type.

Beyond this high-profile accident, air data system anomalies have been reported in numerous airworthiness directives and safety recommendations issued by aviation authorities. Pitot probe icing has prompted mandatory replacement programs across multiple aircraft types operated by major carriers worldwide. Persistent airspeed discrepancies attributed to calibration deviations or installation errors have led to maintenance investigations and operational limitations in a variety of service environments [6], [11].

These cases collectively demonstrate that the fault types selected for evaluation in this thesis — constant bias and frozen signal — are not merely theoretical constructs but reflect real anomaly classes with documented operational consequences. The fact that modern transport-category aircraft incorporate multiple redundant air data channels and sophisticated cross-comparison logic has not eliminated these fault mechanisms entirely. This underscores the continued relevance of structured, quantitative evaluation of even the simplest classical detection methods, as it allows their capabilities and limitations to be understood precisely and communicated clearly in an engineering context.

Furthermore, the incidents cited above illustrate a fundamental challenge in air data fault detection: the difficulty of distinguishing between a genuine sensor fault and transient environmental effects such as atmospheric turbulence, icing, or wake encounter. This ambiguity is directly reflected in the false alarm problem examined in this thesis, where noise-induced signal fluctuations can trigger detection logic in the absence of any true fault. Understanding the baseline behavior of unfiltered classical detection methods under controlled noise conditions, as conducted in this thesis, is a necessary step toward designing monitoring logic that is both sensitive enough to detect real faults and robust enough to reject environmental disturbances.

1.4. Classical Fault Detection Methods in Avionics

Fault detection in avionics systems aims to identify abnormal sensor behavior while minimizing false alarms and maintaining system stability. In certified aircraft systems, monitoring strategies are typically designed to be deterministic, transparent, and verifiable. As a result, classical fault detection techniques remain widely implemented in practical applications [9], [10], [15].

Classical detection methods generally fall into several categories, including threshold-based monitoring, rate-of-change analysis, residual-based approaches, and redundancy comparison logic. While more advanced techniques such as model-based observers and data-driven methods have been proposed in research contexts, certified airborne systems often rely on simpler mechanisms due to certification constraints and robustness considerations [16].

1.4.1. Threshold-Based Monitoring

Threshold-based monitoring is one of the most fundamental and widely implemented fault detection techniques in avionics systems. In this approach, the measured signal is continuously compared against predefined acceptable limits at each sampling instant. If the signal violates any of the defined conditions, a fault condition is declared and a binary flag is raised [9].

Thresholds may be defined based on several engineering criteria. Physical feasibility thresholds enforce minimum and maximum allowable values for the measured parameter. For airspeed, these correspond to operational bounds derived from aircraft aerodynamic and structural limitations. For transport-category aircraft, indicated airspeed is constrained from above by the structural design speed and from below by the minimum control and stall speeds. Operational thresholds may additionally reflect procedural limits defined by the aircraft manufacturer or regulatory authorities [7], [8].

A second class of thresholds addresses the dynamic behavior of the signal rather than its absolute magnitude. Instantaneous change limits, sometimes referred to as jump thresholds, flag sample-to-sample differences that exceed physically plausible rates of change. In airspeed monitoring, such limits exploit the fact that transport-category aircraft cannot accelerate or decelerate instantaneously due to their inertia. An airspeed change of several tens of knots within a single sampling interval of 0.1 seconds is aerodynamically impossible under normal

flight conditions and therefore indicates a sensor fault or data integrity issue rather than a real flight event [9], [11].

The primary engineering advantage of threshold-based monitoring lies in its deterministic and transparent logic. The decision rule can be formally verified, the threshold values can be directly justified from physical or operational specifications, and the implementation requires minimal computational resources. These properties make threshold-based monitoring particularly attractive in certified avionics software, where traceability of design decisions and formal verification of logic are mandatory requirements under standards such as DO-178C [14].

However, threshold-based monitoring has well-recognized limitations. Its effectiveness is fundamentally dependent on the fault magnitude relative to the defined limits. Large abrupt deviations that push the signal outside the acceptable envelope are reliably detected, often within a single sampling interval. In contrast, subtle systematic deviations — such as small constant biases resulting from gradual sensor drift or minor calibration errors — may remain entirely within the operational bounds and therefore pass undetected for extended periods. In such cases, the measured value is numerically plausible but physically incorrect, and threshold logic has no mechanism to identify this discrepancy without reference to an independent source of truth [9].

A further limitation of threshold-based monitoring is its complete structural inability to detect frozen signal conditions under realistic noise levels. When a sensor output becomes stuck at a constant value, the signal remains within the absolute bounds and produces sample-to-sample differences near zero — well below any physically motivated jump threshold. The frozen condition is therefore invisible to threshold logic unless the frozen value itself happens to fall outside the absolute airspeed limits, which in practice is unlikely for a fault occurring during normal cruise or dynamic flight phases. This structural gap is one of the primary motivations for complementing threshold monitoring with rate-of-change analysis, as examined in this thesis.

1.4.2. Rate-of-Change Monitoring

Rate-of-change monitoring evaluates the temporal derivative of the measured signal rather than its absolute magnitude. The discrete first difference between consecutive samples

is calculated and compared against expected dynamic limits [9]. This approach is motivated by the observation that physical systems, including aircraft airspeed dynamics, exhibit bounded rates of change under normal operating conditions. Deviations from expected temporal behavior — either in the direction of excessive variation or abnormal absence of variation — may indicate sensor malfunction.

In the context of airspeed monitoring, rate-of-change analysis addresses two fundamentally different classes of abnormal behavior. The first class involves excessively rapid changes that exceed physically plausible acceleration limits for the aircraft. Such behavior may result from sensor data corruption, digital word errors, or signal processing faults that produce unrealistic sample-to-sample jumps. The detection condition is formally equivalent to the jump threshold used in threshold-based monitoring, and in practice the two methods share this detection logic [11].

The second and more distinctive capability of rate-of-change monitoring is the detection of frozen or stuck signals. When a sensor output stops updating, the discrete derivative approaches zero and remains near zero for as long as the fault persists. During flight phases where the true airspeed is changing — such as climb and descent — the sustained absence of signal variation is physically implausible and constitutes a detectable anomaly. By accumulating consecutive samples where the rate of change falls below a minimum threshold and declaring a fault only after a predefined confirmation duration has elapsed, the monitoring logic correctly identifies frozen behavior while suppressing false alarms from isolated noise dips [9].

The confirmation window, also referred to as the freeze hold time, is a critical design parameter. A shorter window enables faster detection but increases susceptibility to false alarms from momentary low-rate segments caused by measurement noise or smooth flight conditions. A longer window provides greater robustness against false alarms at the cost of increased detection delay. In the implementation adopted for this thesis, a hold time of five seconds was selected, reflecting a balance between detection responsiveness and noise robustness under the modeled noise conditions. In a certified avionics context, this parameter would require formal justification based on operational safety requirements and analysis of the maximum tolerable detection delay for the fault type in question.

An important consideration in rate-of-change monitoring is phase awareness. During cruise flight, airspeed is nominally constant, and the rate of change is therefore naturally near zero for extended periods under normal conditions. Applying freeze detection logic during cruise would generate continuous false alarms on fault-free signals. For this reason, the freeze detection condition in this thesis is restricted to climb and descent phases, where a sustained near-zero rate of change in the airspeed signal is genuinely anomalous. This phase-awareness mechanism illustrates a general principle in avionics monitoring design: detection logic must account for the normal operational context of the monitored signal in order to achieve meaningful sensitivity without unacceptable false alarm rates [9], [11].

1.4.3. Residual-Based Monitoring

Residual-based methods represent a more sophisticated class of classical fault detection techniques. In this framework, a mathematical model of the monitored system is used to generate an expected or predicted value of the sensor output. The residual is defined as the difference between the measured value and the model prediction. Under fault-free conditions, the residual should remain small and statistically consistent with measurement noise. A significant and sustained residual deviation indicates that the measured signal no longer conforms to the model, which may be interpreted as evidence of a sensor fault [9], [10].

The residual-based approach is particularly effective for detecting systematic biases of arbitrary magnitude, including small deviations that remain within absolute threshold limits. Because the detection is based on deviation from a model prediction rather than from a fixed absolute bound, even subtle offsets can be identified if the model is sufficiently accurate and the noise level is low enough to resolve the deviation statistically.

In air data systems, residual-based monitoring may take several forms. Analytical redundancy methods use aerodynamic models or inertial reference data to estimate expected airspeed and compare it against the measured value [4], [5]. Cross-channel comparison, discussed separately in Section 1.4.4, can also be interpreted as a residual-based method in which the residual is computed between two independent sensor measurements rather than between a measurement and a model.

The primary limitation of residual-based monitoring is its dependence on model accuracy. If the model does not adequately represent the true system dynamics — due to

simplifying assumptions, unmodeled atmospheric effects, or off-nominal flight conditions — the residual may exhibit spurious deviations that trigger false alarms. Conversely, if the model is biased in the same direction as the fault, the residual may remain artificially small, delaying or preventing detection. For these reasons, residual-based methods require careful model validation and are generally more complex to implement and certify than threshold or rate-of-change approaches [9], [10].

For the scope of this thesis, residual-based monitoring is not implemented, as it would require an additional system model or redundant sensor that falls outside the defined single-channel evaluation framework. However, the technique is recognized as a natural extension of the present work and is discussed further in the context of future research directions in Section 4.7.

1.4.4. Redundancy and Cross-Comparison Logic

Transport-category aircraft are designed with multiple independent air data channels as a fundamental safety principle. Typically, separate pitot-static systems supply data to independent Air Data Computers assigned to the captain, first officer, and a standby channel [2], [3]. Cross-comparison logic continuously evaluates the agreement between these channels. If the difference between any two channels exceeds a predefined tolerance, the discrepant channel may be flagged as unreliable and isolated from downstream systems.

From a fault detection perspective, cross-comparison logic provides a form of hardware redundancy that is structurally complementary to the analytical methods discussed above. It does not require a mathematical model of the system, and it is effective for detecting any fault that causes one channel to diverge from the others — including biases of arbitrary magnitude, frozen signals, and signal dropout conditions. The detection sensitivity is limited only by the channel comparison threshold and the statistical independence of the measurement noise between channels [2], [9].

However, cross-comparison logic has inherent limitations. It requires a minimum of three independent channels to unambiguously identify which channel is faulty, since a two-channel system can detect a discrepancy but cannot determine which channel is incorrect without additional logic or a casting vote mechanism. Furthermore, in rare scenarios involving common-cause failures — such as simultaneous icing of multiple pitot probes, as occurred in

the AF447 accident — all channels may be affected simultaneously, rendering cross-comparison logic ineffective [13].

Since this thesis focuses on the evaluation of monitoring methods applied to a single simulated airspeed channel, redundancy comparison is outside the defined scope. The single-channel constraint was selected deliberately to isolate the performance characteristics of threshold and rate-of-change logic in a controlled environment, without the confounding influence of inter-channel agreement dynamics.

1.4.5. Rationale for Selected Detection Methods

Among the classical techniques reviewed in this section, threshold-based monitoring and rate-of-change analysis were selected for implementation and evaluation in this thesis. This selection is justified by several considerations.

First, both methods operate on a single sensor channel and require no external model or redundant measurement source, which is consistent with the defined scope of the research. Second, they represent structurally complementary approaches: threshold monitoring targets absolute magnitude violations, while rate-of-change monitoring targets temporal behavior anomalies. Evaluating them together therefore provides insight into both dimensions of classical detection performance. Third, both methods are computationally simple, deterministic, and directly traceable to physical or operational reasoning, making them representative of the monitoring logic actually employed in certified avionics systems [9], [14]. Fourth, the two fault types selected for simulation — constant bias and frozen signal — are specifically chosen to exercise the detection capabilities and expose the structural limitations of each method in a controlled and unambiguous manner.

By maintaining this focused scope, the thesis enables a clear, quantitative, and reproducible assessment of classical detection performance without introducing the additional complexity of model-based or multi-channel methods. The results obtained under this framework provide a meaningful baseline against which more advanced techniques may be compared in future work.

1.5. Performance Metrics in Fault Detection

The evaluation of fault detection methods requires clearly defined performance metrics. In practical avionics applications, detection mechanisms must balance sensitivity to abnormal behavior with robustness against noise and operational variability. Excessive sensitivity may lead to false alarms, while insufficient sensitivity may delay or miss fault detection entirely [9], [10].

For the scope of this thesis, performance evaluation focuses on three primary metrics: detection delay, missed detection, and false alarm occurrence. These metrics are commonly used in fault detection analysis and provide a direct and interpretable measure of monitoring effectiveness.

1.5.1. Detection Delay

Detection delay represents the time interval between the occurrence of a fault and the moment when the detection logic identifies the abnormal condition.

The detection delay Δt is defined as:

$$\Delta t = t_d - t_f \quad (1.3)$$

where: t_d – is fault detected time

t_f – is fault occurs time

Minimizing detection delay is desirable, particularly for faults that may influence flight control or pilot awareness. However, overly aggressive detection thresholds may reduce delay at the cost of increased false alarms.

In the case of frozen signal faults, detection delay depends on the monitoring logic's ability to recognize absence of expected signal variation. For bias faults, detection delay may depend on whether the deviation exceeds predefined threshold limits.

1.5.2. Missed Detection

A missed detection occurs when a fault is present in the signal but is not identified by the monitoring logic within the observation period.

Missed detections are particularly critical because they represent failure of the detection mechanism to respond to abnormal behavior. In practical avionics systems, undetected faults may propagate incorrect data to downstream systems.

In the context of this thesis, a missed detection is recorded when a simulated fault is introduced but no-fault flag is triggered by the detection logic during the analysis window.

1.5.3. False Alarm

A false alarm occurs when the detection logic indicates a fault condition despite the absence of any actual fault in the signal.

False alarms are undesirable because they may:

- Reduce confidence in monitoring systems,
- Increase pilot workload,
- Trigger unnecessary maintenance actions.

In simulation-based evaluation, false alarms can be quantified by counting instances where detection logic is activated during fault-free segments of the signal.

Noise level and signal variability strongly influence false alarm rates. For example, excessive noise measurement may cause rate-of-change thresholds to be exceeded unintentionally. Therefore, sensitivity settings must be carefully selected to balance detection reliability and robustness.

1.5.4. Trade-Off Between Sensitivity and Robustness

Fault detection inherently involves a trade-off between rapid detection and resistance to false alarms. Lower detection thresholds may result in faster identification of subtle faults but increase sensitivity to noise. Conversely, conservative thresholds may reduce false alarms but delay detection.

In certified avionics systems, conservative design choices are often favored to ensure stable and predictable behavior. Evaluating this trade-off under controlled simulated conditions allows systematic assessment of how classical detection methods perform under different noise and operational scenarios.

1.5.5. Evaluation Approach in This Thesis

Based on the metrics discussed, the evaluation methodology in this thesis will:

- Measure detection delay for each fault scenario.
- Record instances of missed detection.
- Quantify false alarm occurrences in fault-free segments.
- Compare results across different noise levels and flight phases.

This structured evaluation framework provides a clear basis for comparing threshold-based and rate-of-change monitoring methods without introducing unnecessary complexity.

1.6. Summary of Literature and Research Gap

This chapter reviewed the fundamental principles and operational context of aircraft air data systems, with emphasis on airspeed measurement and processing. The structure of the pitot-static system, the role of the Air Data Computer, and the operational significance of indicated airspeed were discussed to establish the technical foundation of the research object.

Common air data sensor failure modes were examined, including pitot tube blockage, static port obstruction, calibration deviations, and electronic disturbances. Particular attention was given to constant bias and frozen signal faults, as these represent distinct and practically relevant abnormal behaviors observed in air data systems.

Classical fault detection approaches used in avionics were then reviewed. Threshold-based monitoring and rate-of-change analysis were identified as widely implemented techniques due to their simplicity, transparency, and suitability for certified airborne systems. Other classical methods, such as residual-based monitoring and redundancy comparison logic, were briefly discussed to provide context, although they are outside the scope of this thesis.

Finally, commonly used performance metrics for fault detection evaluation were presented, including detection delay, missed detection, and false alarm occurrence. These metrics provide a structured basis for assessing monitoring effectiveness under controlled conditions.

Although classical detection techniques are well established in avionics practice, structured quantitative evaluation of their performance under simplified and controlled simulation scenarios remains beneficial for understanding their practical behavior and

limitations. In particular, the influence of the noise measurement and flight phase characteristics on detection delay and false alarm rates warrants systematic analysis.

The regulatory and certification context governing the application of such methods is reviewed in the following section. Chapter 2 then presents the methodology adopted in this thesis for simulating airspeed data, modeling representative fault types, implementing classical detection logic, and evaluating detection performance.

1.7. Regulatory and Certification Context for Air Data Monitoring

The design and validation of fault detection mechanisms in certified aircraft systems are governed by a framework of regulatory requirements and technical standards. Understanding this regulatory context is important for interpreting the scope and limitations of the evaluation conducted in this thesis and for situating classical fault detection methods within the broader landscape of airworthiness engineering.

The primary regulatory framework for large transport-category aircraft in Europe is established by the European Union Aviation Safety Agency through the Certification Specifications for Large Aeroplanes, known as CS-25 [17]. CS-25 defines airworthiness requirements covering structural integrity, systems and equipment, flight performance, and continued airworthiness. Within the system's requirements, provisions address the reliability and integrity of flight-critical systems including air data sensors and their associated monitoring functions. Specifically, CS-25 requires that failures of systems whose loss could affect safe flight and landing be shown to have acceptably low probability of occurrence, and that the effects of failures be mitigated through design, redundancy, or monitoring mechanisms.

The Federal Aviation Administration applies analogous requirements through the Federal Aviation Regulations Part 25, which is harmonized with CS-25 in most substantive respects [7]. Together, these regulations define the safety objectives that air data monitoring systems must satisfy but do not prescribe specific technical implementations. The choice of monitoring method — whether threshold-based, rate-of-change, residual-based, or redundancy comparison — is left to the system designer, provided the chosen approach can be shown to meet the required safety objectives through analysis, simulation, or flight test evidence.

Environmental qualifications of airborne equipment, including pressure sensors and Air Data Computers, is governed by the RTCA standard DO-160, which specifies test conditions for temperature, vibration, humidity, electromagnetic compatibility, and other environmental

factors [14]. While DO-160 does not directly address fault detection logic, it establishes the environmental envelope within which sensor hardware must operate reliably. The performance boundaries assumed in fault modeling and detection design must therefore be consistent with the environmental conditions characterized by DO-160 qualification testing.

For software aspects of airborne systems, including the detection algorithms implemented in Air Data Computers, the applicable standard is DO-178C, which defines the software development assurance levels and verification requirements for avionics software. Detection logic implemented in safety-critical avionics software is typically required to meet Development Assurance Level B or Level A, depending on the severity of the failure condition associated with a missed detection or false alarm. These levels impose rigorous requirements on requirements traceability, code coverage, and formal verification that go significantly beyond the scope of a simulation-based academic evaluation [9].

The Monte Carlo statistical evaluation methodology adopted in this thesis is consistent with the analytical approach recommended in AC 25.1309, the advisory circular that provides guidance on the means of compliance with the system safety requirements of CS-25 and FAR Part 25. AC 25.1309 accepts probabilistic analysis of failure rates and detection performance as a valid means of demonstrating compliance, provided the analysis is conducted with appropriate rigor and under representative conditions. While the simulated environment used in this thesis is a simplified abstraction of real flight conditions, the structured experimental design and quantitative performance metrics employed are aligned with the general principles of this advisory material.

It is important to note that the thresholds and parameters used in the detection logic implemented in this thesis were selected on the basis of physical reasoning and engineering judgment rather than formal optimization against certified aircraft specifications. The values adopted — including the airspeed bounds, rate limits, and freeze confirmation window — are broadly representative of parameters found in the literature and consistent with operational ranges for transport-category aircraft [7], [9], [11]. However, applying these methods in a certified avionics context would require formal threshold derivation based on aircraft-specific performance data, environmental qualification test results, and safety analysis conducted in accordance with the applicable certification standards.

Understanding this regulatory framework clarifies the position of the present thesis within the broader field of avionics fault detection research. The work does not attempt to

demonstrate regulatory compliance or to produce certified monitoring software. Rather, it provides a structured quantitative evaluation of the fundamental performance characteristics of classical detection methods under controlled conditions, which forms a necessary analytical foundation for any subsequent development aimed at certified implementation.

2. METHODOLOGY

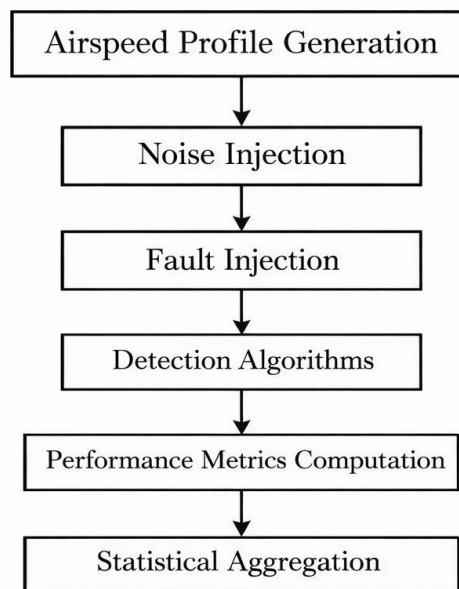
2.1. Methodological Framework

The objective of this chapter is to describe the simulation-based framework developed to evaluate classical fault detection methods applied to airspeed sensor data. The methodology is structured to ensure reproducibility, controlled variability, and statistical robustness.

The evaluation framework consists of five sequential stages:

1. Generation of representative airspeed time-series profiles.
2. Addition of measurement noise to simulate realistic sensor behavior.
3. Injection of predefined fault types.
4. Implementation of classical detection algorithms.
5. Quantitative performance evaluation through Monte Carlo analysis.

The complete workflow is illustrated in Picture 2.1.



Pic. 2.1. Overview of the Simulation and Evaluation Framework [Author's own work]

2.2. Airspeed Signal Simulation

2.2.1. Modelling Assumptions

To maintain analytical focus on fault detection rather than aerodynamic modeling complexity, the airspeed signal is directly generated as indicated airspeed (IAS). The pitot-static pressure equations are not explicitly modeled.

This simplification is justified because:

- Detection logic operates on airspeed signal level.
- The objective is comparative evaluation of monitoring performance.
- Fault characteristics are signal-based, not pressure-model-based.

Such signal-level modeling is common in FDI benchmarking studies [9], [11].

2.2.2. Flight Profile Structure

The simulated flight profile consists of three phases:

- Climb
- Cruise
- Descent

Total simulation duration:

$$T = 600 \text{ seconds} \quad (2.1)$$

Sampling frequency:

$$\Delta t = 0.1s \text{ and } fs = 10Hz \quad (2.2)$$

This provides sufficient temporal resolution for dynamic evaluation.

2.2.3. Piecewise Airspeed Profile Definition

The clean airspeed profile $V(t)$ is defined piecewise:

- Climb phase ($0 \leq t < t_1$):

$$V(t) = V_0 + \alpha t \quad (2.3)$$

- Cruise phase ($t_1 \leq t < t_2$):

$$V(t) = V_c \quad (2.4)$$

- Descent phase ($t_2 \leq t \leq T$):

$$V(t) = V_c - \beta(t - t_2) \quad (2.5)$$

where: $V_0 = 140 \text{ kt}$

$V_c = 250 \text{ kt}$

$V_f = 160 \text{ kt}$

α and β are chosen to produce realistic gradual acceleration/deceleration.

These values represent typical transport-category operational ranges [7], [12].

Optional low-amplitude sinusoidal perturbations may be included to reflect minor operational variability.

2.3. Noise Modeling

Measurement noise is modeled as additive white Gaussian noise:

$$V_n(t) = V(t) + \varepsilon(t) \quad (2.6)$$

where:

$$\varepsilon(t) \sim N(0, \sigma^2) \quad (2.7)$$

Three noise levels are considered:

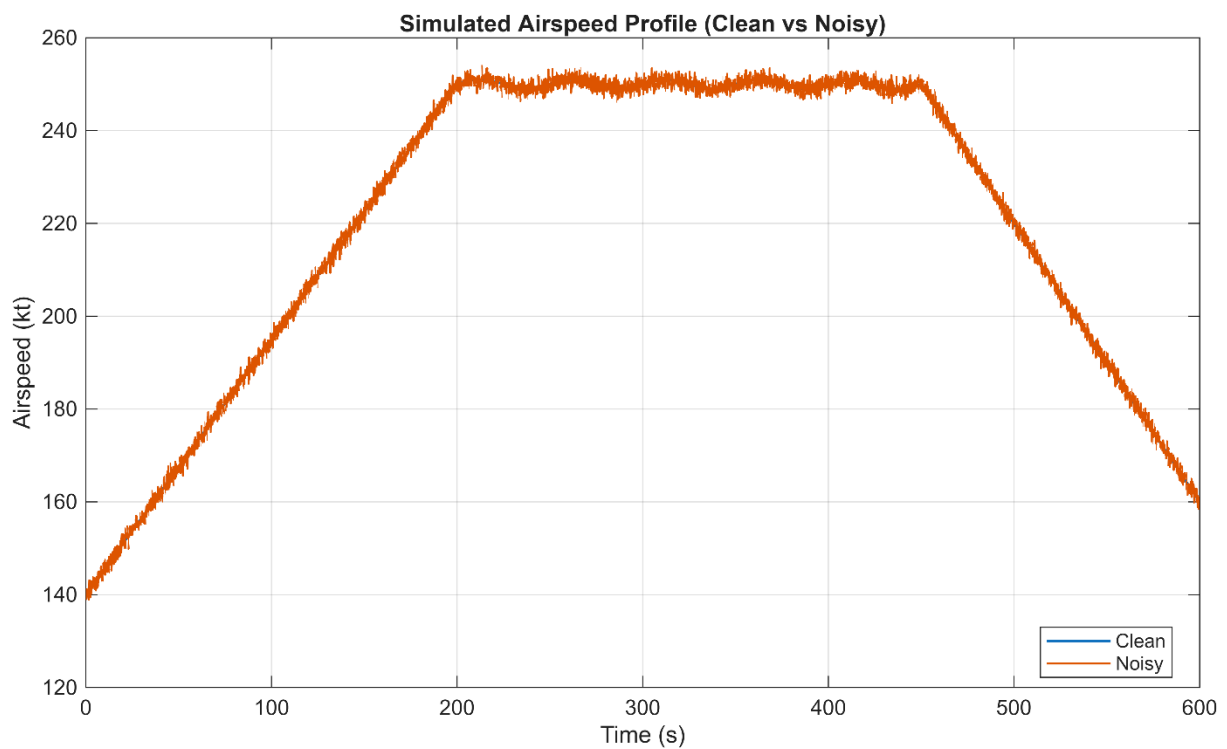
Table 2.1

Noise Levels	
Level	σ (kt)
Low	0.5
Medium	1.0
High	2.0

Gaussian noise modeling is standard in sensor performance simulation [9], [10], [14].

Noise is applied prior to fault injection to emulate realistic measurement disturbance.

An example of the generated clean airspeed profile together with its noisy counterpart is illustrated in Picture 2.2. The figure demonstrates the piecewise climb–cruise–descent structure and the effect of additive measurement noise.



Pic. 2.2. Example Simulated Airspeed Profile with Clean and Noisy Signals Across Climb, Cruise, and Descent Phases [Author's own work]

2.4. Fault Modeling

Two fault types are introduced.

2.4.1. Constant Bias Fault

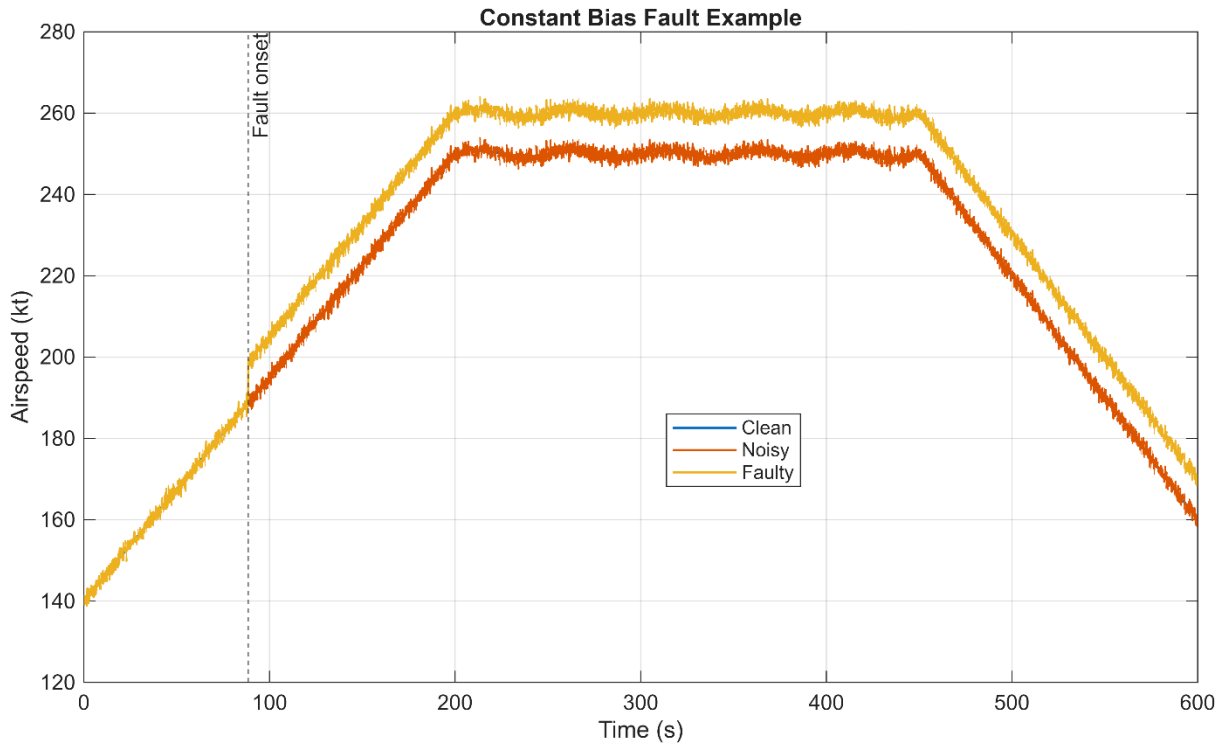
A constant bias fault is modeled as:

$$V_b(t) = V_n(t) + \Delta V \quad (2.8)$$

for $t \geq t_f$,

where: $\Delta V = +10 \text{ kt}$

An example of the simulated bias fault applied during the climb phase is shown in Picture 2.3. The figure illustrates the constant offset introduced after the fault onset time while preserving the dynamic characteristics of the signal.



Pic. 2.3. Example of simulated constant bias fault applied during the climb phase under medium noise conditions. [Author's own work]

This magnitude represents a noticeable but non-catastrophic deviation.

Bias preserves dynamic characteristics but shifts to absolute magnitude.

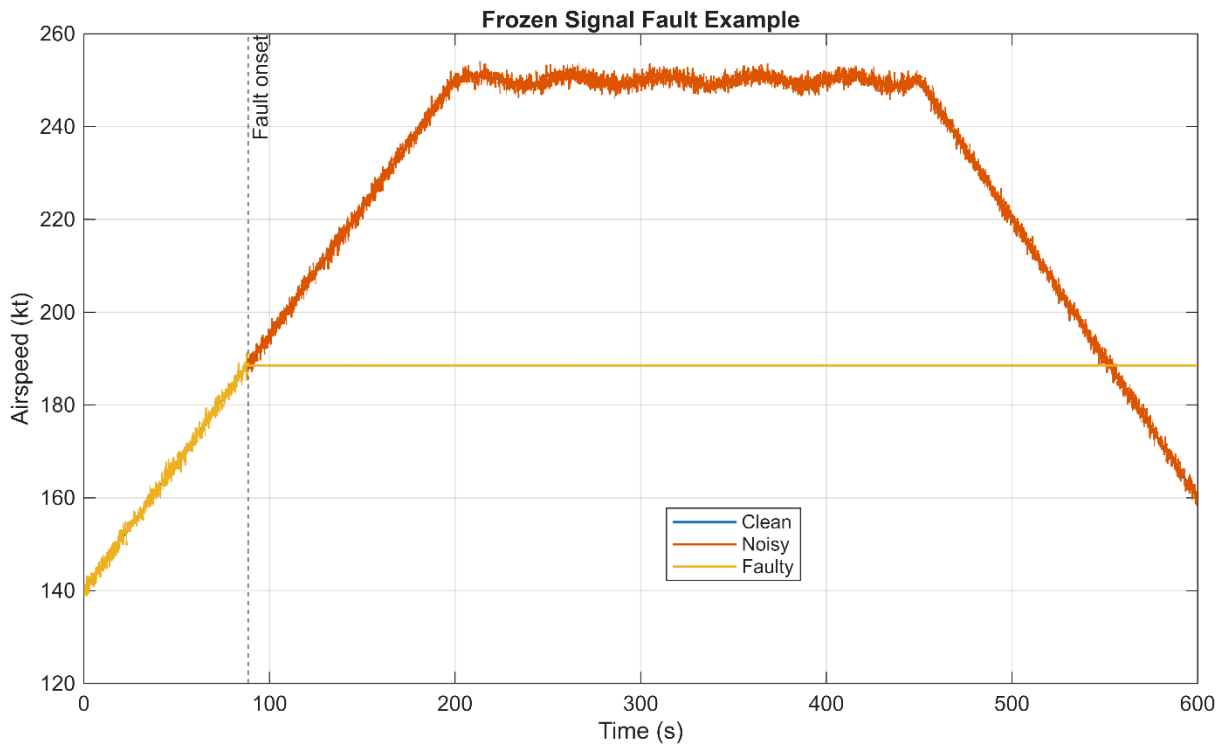
2.4.2. Frozen Signal Fault

A frozen signal fault is modeled as:

$$V_f(t) = V_n(t_f) \quad (2.9)$$

for $t \geq t_f$,

An example of the frozen signal fault is illustrated in Picture 2.4. After the fault onset, the airspeed signal remains constant despite changes in the underlying flight profile.



Pic. 2.4. Example of simulated frozen signal fault applied during the climb phase under medium noise conditions. [Author's own work]

This simulates pitot blockage leading to constant output.

Fault onset time t_f is randomized within dynamic phases (climb/descent).

2.5. Fault Detection Implementation

This section defines the exact detection logic implemented in MATLAB. The methods are intentionally simple and deterministic.

2.5.1. Threshold-Based Detection

Threshold-based monitoring compares the measured airspeed signal against predefined acceptable limits.

Two threshold conditions are implemented:

a) Absolute Airspeed Limits

A fault is declared if:

$$V(t) < V_{\min} \text{ or } V(t) > V_{\max} \quad (2.10)$$

where: $V_{\min} = 60 \text{ kt}$

$V_{\max} = 400 \text{ kt}$

These limits represent physically plausible operational bounds for transport-category aircraft.

b) Maximum Allowed Instantaneous Change

$$|V(k) - V(k - 1)| > J_{\max} \quad (2.11)$$

where: $J_{\max} = 15 \text{ kt/s}$

Binary flag:

$F_T(k) = 1$ if violation occurs

$F_T(k) = 0$ otherwise

Threshold-based monitoring is widely used in avionics due to simplicity [9], [17].

2.5.2. Rate-of-Change Monitoring

Rate-of-change monitoring evaluates the dynamic behavior of the signal.

The discrete derivative is computed as:

$$\dot{V}(k) = \frac{V(k) - V(k - 1)}{\Delta t} \quad (2.12)$$

where Δt is the sampling interval.

Two abnormal behaviors are monitored:

a) Excessive Rate

$$|\dot{V}(k)| > R_{\max} \quad (2.13)$$

where: $R_{\max} = 20\text{kt/s}$

b) Frozen Signal Detection

If:

$$|\dot{V}(k)| < R_{\min} \quad (2.14)$$

for a continuous duration exceeding T_{freeze} ,

Parameters:

$$R_{\min} = 0.2\text{kt/s}$$

$$T_{\text{freeze}} = 5\text{seconds}$$

Decision Logic

$F_R(k) = 1$ if abnormal condition satisfied.

Rate monitoring is commonly applied in sensor validation logic [9], [11].

2.6. Experimental Design

This section defines how simulations are conducted.

2.6.1. Scenario Matrix

Each experiment consists of combinations of:

- 2 flight phases (climb, descent)
- 3 noise levels (low, medium, high)
- 2 fault types (bias, frozen)

Total scenarios:

$$2 \times 3 \times 2 = 12 \quad (2.15)$$

2.6.2. Fault Onset Time

Fault onset time t_f is randomized within predefined intervals:

- Climb: 30%–60% of climb duration
- Descent: 30%–60% of descent duration

Randomization ensures statistical robustness and prevents overfitting detection to a fixed location.

2.6.3. Number of Simulation Runs

For each scenario:

$$N = 50 \text{ Monte Carlo runs} \quad (2.16)$$

Total simulations:

$$12 \times 50 = 600 \quad (2.17)$$

Randomization:

$$t_f \in [30\%, 60\%] \text{ of phase duration} \quad (2.18)$$

Randomization ensures statistical generality [18].

2.7. Performance Evaluation Procedure

This section defines exactly how results are computed.

2.7.1. Detection Delay Calculation

For each run:

$$\Delta t = t_d - t_f \quad (2.19)$$

Average detection delay per scenario:

$$\bar{\Delta t} = \frac{1}{N} \sum_{i=1}^N \Delta t_i \quad (2.20)$$

2.7.2. Missed Detection Rate

$$P_{miss} = \frac{\text{Number of undetected faults}}{N} \quad (2.21)$$

2.7.3. False Alarm Rate

False alarms are counted during fault-free segments.

False alarm probability:

$$P_{FA} = \frac{\text{Number of false detections}}{\text{Total fault-free observation windows (N)}} \quad (2.22)$$

2.8. Statistical Post-Processing

Results are aggregated using Python:

- Grouped means
- Scenario comparison tables
- Visualization plots

This separation improves reproducibility and computational transparency.

The Python scripts used for statistical aggregation and visualization are provided in Appendices 10 and 11. Execution instructions and software requirements are provided in Appendix 12.

3. RESULTS

3.1. Overview of Simulation Campaign

A total of 600 simulation runs were performed using a full factorial design comprising two fault types (bias and freeze), three noise levels (low, medium, and high), and two flight phases (climb and descent). For each configuration, 50 Monte Carlo repetitions were executed with randomized fault onset times.

Detection performance was evaluated using mean detection delay, missed detection rate, and mean false alarm event count per run.

3.2. Detection Performance for Bias Faults

Table 3.1 summarizes detection performance for constant bias faults.

Table 3.1

Mean Detection Metrics for Bias Faults

Noise	Phase	T Delay (s)	T Miss Rate	R Delay (s)	R Miss Rate
Low	Climb	0.00	0.00	0.00	0.00
Low	Descent	0.00	0.00	0.00	0.00
Medium	Climb	0.00	0.00	0.00	0.00
Medium	Descent	0.00	0.00	0.00	0.00
High	Climb	0.00	0.00	0.00	0.00
High	Descent	0.03	0.00	0.03	0.00

The results indicate that both threshold-based and rate-of-change detection methods successfully identified all injected bias faults across all noise levels and flight phases. The missed detection rate was zero in every scenario.

Mean detection delay was approximately zero seconds, indicating that bias faults were detected immediately at fault onset. A slight non-zero average delay (0.03 s) observed under high-noise descent conditions is attributed to discrete-time sampling effects and does not represent meaningful performance degradation.

These findings confirm that classical monitoring approaches are highly effective in detecting abrupt bias deviations in airspeed measurements.

3.3. Detection Performance for Freeze Faults

Table 3.2 summarizes detection performance for frozen signal faults.

Table 3.2

Mean Detection Metrics for Freeze Faults

Noise	Phase	T Delay (s)	T Miss Rate	R Delay (s)	R Miss Rate
Low	Climb	—	1.00	5.00	0.00
Medium	Climb	—	1.00	5.00	0.00
High	Climb	0.00	0.78	3.90	0.00
Low	Descent	—	1.00	4.99	0.00
Medium	Descent	—	1.00	4.99	0.00
High	Descent	0.00	0.86	4.30	0.00

The threshold-based method exhibited limited effectiveness in detecting freeze faults. Under low and medium noise conditions, the missed detection rate reached 100% for both climb and descent phases, indicating complete failure to identify the frozen signal condition.

Under high noise conditions, partial detection occurred (miss rates of 78% in climb and 86% in descent). These detections are attributed to noise-induced rate fluctuations rather than intentional freeze recognition logic.

In contrast, the rate-of-change method successfully detected all freeze faults across all tested scenarios, with zero missed detections.

Mean detection delay for freeze faults was approximately 5 seconds under low and medium noise conditions. This corresponds directly to the predefined freeze detection window implemented in the monitoring logic. Under high noise conditions, the average delay was slightly reduced (3.9–4.3 s), reflecting earlier threshold crossings due to noise perturbations.

These results demonstrate that rate-of-change monitoring is significantly more effective than static threshold checks for identifying frozen airspeed signals.

3.4. False Alarm Analysis

False alarm performance was evaluated by counting the number of 0-to-1 transitions of the detection flag during fault-free intervals.

Table 3.3 presents mean false alarm events per run for both detection methods.

Table 3.3

Mean False Alarm Events per Run

Fault	Noise	Phase	T FA Mean	R FA Mean
Bias	Low	Climb	0.00	0.00
Bias	Low	Descent	0.00	0.00
Bias	Medium	Climb	3.64	3.64
Bias	Medium	Descent	22.18	22.18
Bias	High	Climb	95.42	95.42
Bias	High	Descent	576.88	576.88
Freeze	Low	Climb	0.00	0.00
Freeze	Low	Descent	0.00	0.00
Freeze	Medium	Climb	3.72	3.72
Freeze	Medium	Descent	21.60	21.60
Freeze	High	Climb	102.02	102.02
Freeze	High	Descent	573.06	573.06

False alarm frequency increased substantially with measurement noise magnitude. Under low noise conditions, false alarms were negligible. However, medium noise introduced occasional spurious detections, particularly during descent. High noise conditions produced significantly elevated false alarm counts, especially in descent phase scenarios.

The similarity between threshold and rate-of-change false alarm values indicates that both methods are comparably sensitive to noise-induced fluctuations under extreme conditions.

3.5. Overall Summary of Results

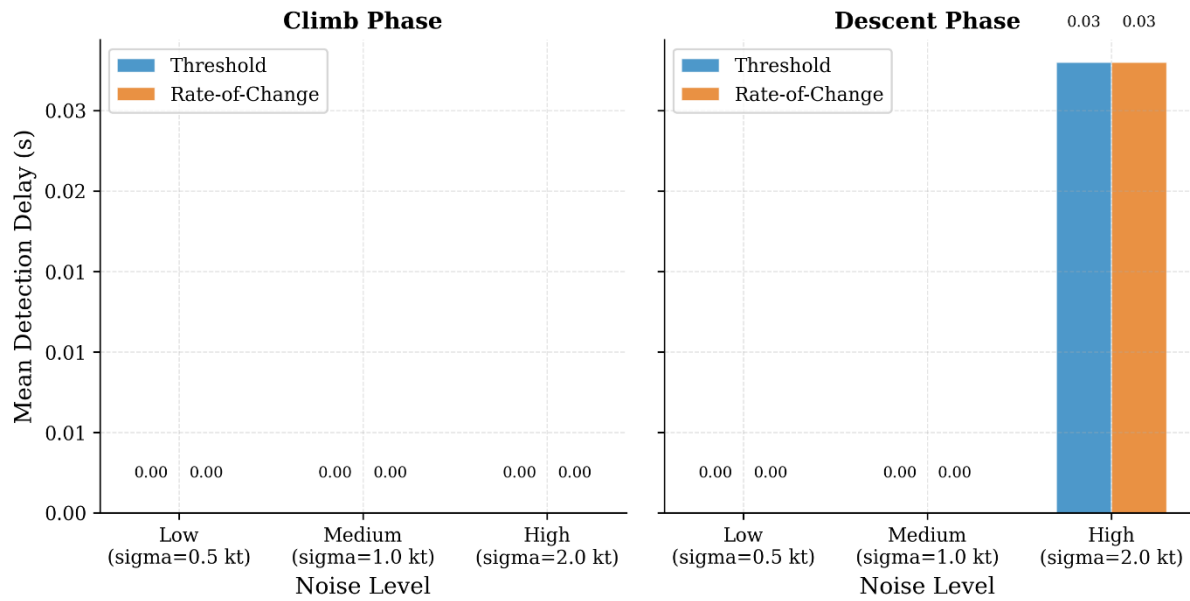
The simulation campaign demonstrates a clear and consistent differentiation in the effectiveness of the two classical detection methods depending on fault type. For constant bias faults, both threshold-based and rate-of-change monitoring achieved complete detection across all tested noise levels and flight phases, with negligible detection delay. This confirms that an abrupt fixed offset of the magnitude considered (+10 kt) is reliably captured by simple absolute limit checks, and that rate-of-change logic provides no additional benefit in this scenario. For frozen signal faults, the results reveal a fundamental limitation of threshold-based monitoring:

under low and medium noise conditions, the method failed entirely, producing a missed detection rate of 100%. Detection only occurred partially under high noise conditions, where noise-induced fluctuations coincidentally triggered the rate threshold rather than any intentional frozen-signal recognition logic. The rate-of-change method, by contrast, detected all freeze faults in every scenario with zero missed detections, at a mean delay governed by the predefined freeze window of 5 seconds. False alarm analysis shows that both methods are comparably sensitive to noise magnitude, with alarm frequency increasing substantially from low to high noise conditions and being particularly pronounced during the descent phase. This phase-dependent behavior is consistent with the longer fault-free observation window available during descent, which naturally accumulates more noise-induced threshold crossings. Taken together, the results indicate that classical detection methods are not universally applicable across fault types: threshold-based monitoring is appropriate for large abrupt deviations but is structurally blind to frozen signal conditions under realistic noise levels, while rate-of-change monitoring provides robust freeze detection at the cost of elevated false alarm sensitivity under high noise. These findings provide a quantitative foundation for the Discussion that follows.

3.6. Visualization of Detection Performance

The tabular results presented in Sections 3.2 through 3.4 are complemented in this section by four figures generated from the Monte Carlo simulation dataset using Python-based post-processing scripts. Each figure was produced from the aggregated results contained in `summary_grouped.csv` and `summary_runs.csv` using the scripts provided in Appendices 10 and 11. The figures provide a visual representation of the key performance trends and support the analytical interpretations developed in the subsequent Discussion chapter.

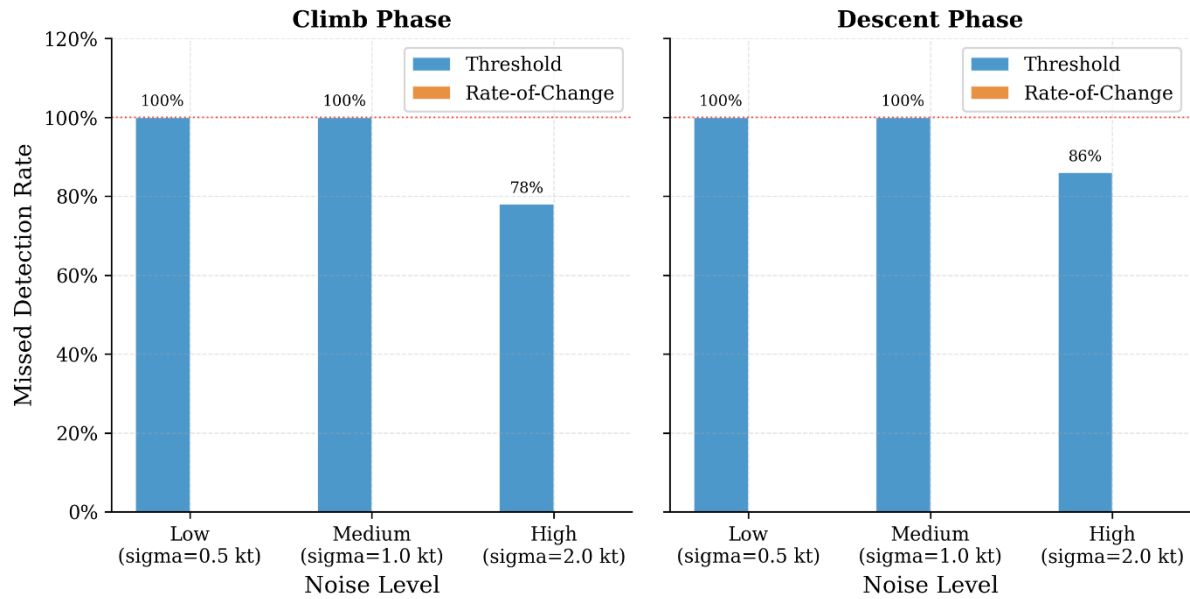
Picture 3.1 presents the mean detection delay for constant bias faults across both flight phases and all three noise levels, comparing the threshold-based and rate-of-change detectors side by side.



Pic. 3.1. Mean detection delay for constant bias faults across noise levels and flight phases, comparing threshold-based and rate-of-change detectors. [Author's own work]

As illustrated in Picture 3.1, detection delay for bias faults is negligible across all tested configurations. The near-zero delay values confirm that a bias of +10 kt is detected within a single sampling interval under all noise conditions. The only non-zero delay of 0.03 s appears under high-noise descent conditions for both detectors simultaneously, consistent with the discrete-time sampling artefact reported in Table 3.1. The visual overlap of the two detector bars across all panels confirms that the threshold and rate-of-change methods provide equivalent performance for this fault type.

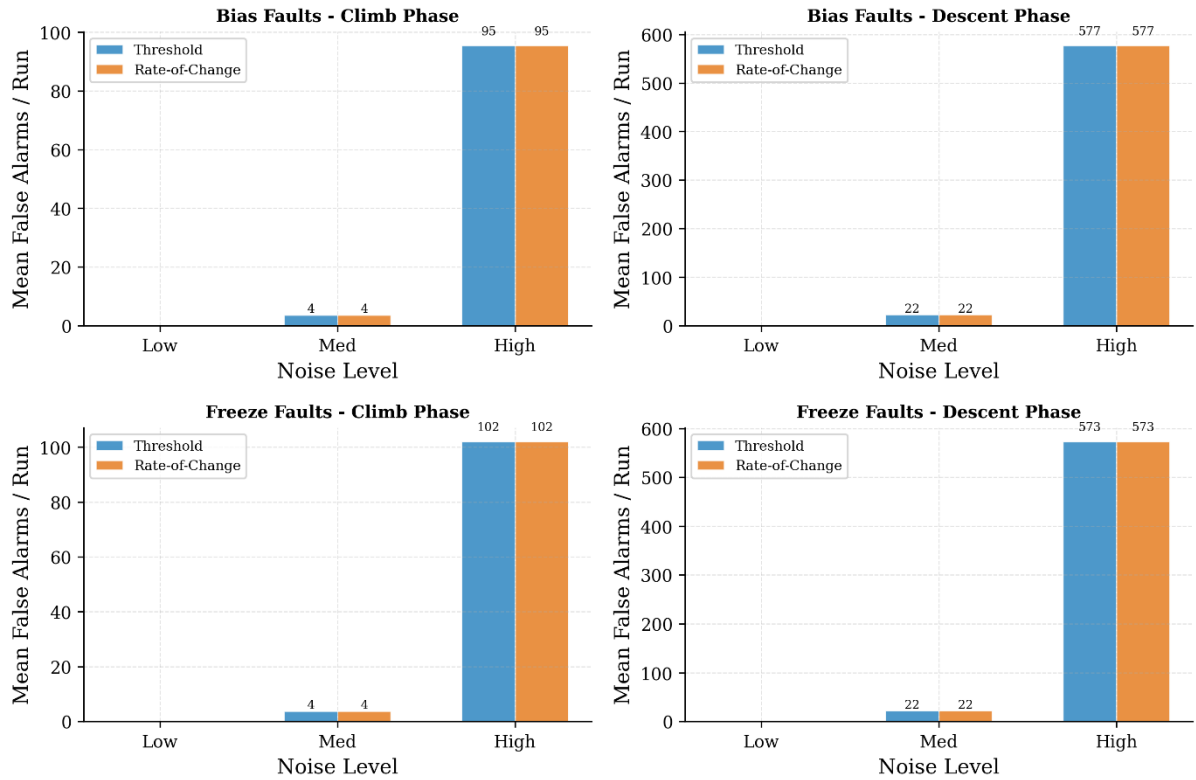
Picture 3.2 presents the missed detection rate for frozen signal faults across both flight phases and noise levels. The rate-of-change detector bars are absent from all panels because the missed detection rate for this method is zero in every scenario.



Pic. 3.2. Missed detection rate for frozen signal faults across noise levels and flight phases. The rate-of-change detector achieved zero missed detections in all scenarios and is not visible in the figure. The red dotted line indicates 100% missed detection rate. [Author's own work]

Picture 3.2 provides the clearest visual illustration of the structural difference between the two detection methods. The threshold detector bars reach 100% missed detection under low and medium noise conditions in both phases, confirming complete failure to identify the frozen signal condition. The partial reduction observed at high noise — to 78% in climb and 86% in descent — reflects noise-induced detections rather than genuine freeze recognition, as discussed in Section 3.3. The complete absence of the rate-of-change detector bars visually confirms zero missed detections across all 300 freeze fault simulation runs evaluated by that method.

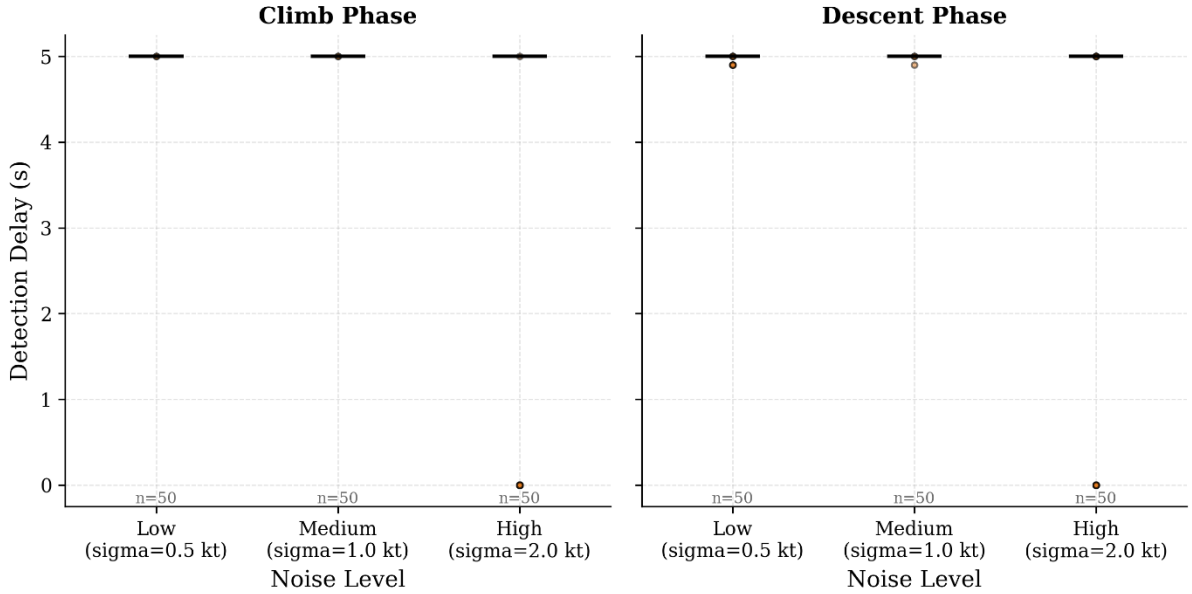
Picture 3.3 presents the mean false alarm event count per run for all scenario combinations, organised across four panels corresponding to the two fault types and two flight phases.



Pic. 3.3. Mean false alarm events per run for all scenario and phase combinations. Blue bars indicate threshold-based detector; orange bars indicate rate-of-change detector. [Author's own work]

The four-panel layout of Picture 3.3 reveals two consistent patterns. First, both detectors produce numerically identical false alarm counts in every panel, confirming the shared instantaneous jump check mechanism described in Section 3.4. Second, the false alarm count escalates sharply from low to high noise in all panels. The descent phase panels exhibit substantially higher counts than their climb counterparts — reaching approximately 577 events per run under high noise for bias fault scenarios — consistent with the longer fault-free observation window available during the descent phase as explained in Section 3.4. Under low noise conditions, false alarm counts are negligible across all scenario combinations.

Picture 3.4 presents the distribution of detection delay for frozen signal faults under the rate-of-change detector across both flight phases, displayed as box plots grouped by noise level.



Pic. 3.4. Distribution of detection delay for frozen signal faults under the rate-of-change detector. Each box represents 50 Monte Carlo runs. The tight clustering at 5 seconds confirms the deterministic nature of the freeze detection window. [Author's own work]

Picture 3.4 reveals a characteristic that the mean values in Table 3.2 alone do not fully convey. Under low and medium noise conditions in both phases, the detection delay is concentrated with extremely low variance at approximately 5 seconds, confirming that the freeze detection window dominates the detection timing and that individual run-to-run variability in fault onset time does not meaningfully influence the delay. Under high noise conditions, the distributions show marginally earlier median detection and slightly wider spread, reflecting occasional noise-induced early crossings of the rate threshold before the full 5-second window elapses. The $n=50$ annotation on each box confirms that all 50 Monte Carlo runs produced a valid detection under the rate-of-change method, regardless of noise level or flight phase — consistent with the zero missed detection rate reported in Table 3.2.

Taken together, the four figures confirm and visually reinforce the primary findings of the simulation campaign. Bias faults are detected immediately by both methods with equivalent performance. Frozen signal faults are reliably detected only by the rate-of-change method, with a consistent and predictable 5-second detection latency. False alarm behavior is driven primarily by noise magnitude and observation window length and is identical between both detectors under the current implementation.

4. DISCUSSION

4.1. Interpretation of Bias Fault Detection Results

The complete detection of constant bias faults by both methods across all noise levels and flight phases is consistent with the fault magnitude selected for this study. A bias of +10 kt shifts the airspeed signal beyond the instantaneous change threshold defined in the monitoring logic, triggering an immediate detection flag at fault onset. This result confirms that threshold-based monitoring is well suited for detecting abrupt, large-magnitude deviations in airspeed data, which aligns with its widespread use in certified avionics architectures where operational limits are clearly defined and physically motivated [9].

The mechanism of detection in this case deserves precise engineering explanation. The instantaneous change condition flags the sample-to-sample difference at the moment of fault onset. Because the bias is injected as a step function — the signal shifts by the full +10 kt in a single sampling interval of 0.1 seconds — the resulting discrete derivative is 100 kt/s at the instant of onset. This value far exceeds the configured jump threshold of 40 kt/s per sample, producing an immediate flag regardless of the noise level. After the initial onset sample, the bias-affected signal resumes its normal dynamic behavior with the offset applied, and no further rate violations occur unless the noise level is high enough to produce additional spurious jump events. This explains why the detection delay is consistently zero or near-zero across all bias fault scenarios: detection occurs at the first sample of the fault, not after any accumulation period.

The slight non-zero average delay of 0.03 seconds observed under high-noise descent conditions for both detectors simultaneously is a discrete-time sampling artefact rather than a genuine detection lag. At a sampling frequency of 10 Hz, the minimum resolvable detection delay is 0.1 seconds, which corresponds to one sampling interval. A mean delay of 0.03 seconds averaged over 50 Monte Carlo runs implies that in the majority of runs detection occurred at the fault onset sample, while in a small number of runs the fault onset fell between two sampling instants such that the first affected sample registered a slightly reduced difference. This is an expected property of discrete-time monitoring systems and does not represent a meaningful performance limitation.

However, it is important to note that this finding is contingent on the bias magnitude exceeding the detection threshold. In operational contexts, smaller systematic biases — for

example, a drift of 2 to 3 kt due to gradual sensor degradation or minor calibration error — would remain within the absolute airspeed limits and would not trigger the instantaneous change check. Such subtle biases represent a known and practically significant limitation of threshold-based monitoring, as noted in the literature [9], [11]. A bias of this magnitude could persist undetected for the entire duration of a flight without triggering any classical monitoring flag, yet it would still constitute a systematic measurement error with potential consequences for flight envelope protection margins. The results of this thesis, while demonstrating complete detection for the selected fault magnitude, should therefore not be interpreted as evidence that threshold monitoring is universally reliable for all bias fault scenarios.

The choice of +10 kt was deliberate in providing a clear, unambiguous test case that produces consistent and interpretable results across all noise levels. A systematic sensitivity study varying bias magnitude from small values below the detection threshold to large values well above it would be a natural and valuable extension of the present work. Such a study would allow the minimum detectable bias to be characterized precisely as a function of noise level and threshold parameters, providing a more complete picture of detection sensitivity boundaries than the single-magnitude evaluation presented here. This direction is discussed further in Section 4.7.

It is also worth observing that both the threshold-based and rate-of-change detectors produced identical detection results for bias faults in every scenario. This is structurally expected, since the detection of a step-change bias relies on the instantaneous jump check, which is implemented identically in both detection functions as defined in the MATLAB code. The rate-of-change method does not provide any additional benefit over the threshold method for this fault type under the current implementation. This equivalence is an important finding in itself: it demonstrates that for abrupt large-magnitude faults, the additional complexity of rate-of-change logic provides no incremental detection capability beyond what a simple threshold check already achieves. The practical implication is that for detecting large bias faults, the simpler threshold method is entirely sufficient, and the rate-of-change method contributes value only through its freeze detection capability.

4.2. Interpretation of Freeze Fault Detection Results

The failure of threshold-based monitoring to detect frozen signal faults under low and medium noise conditions is structurally expected and deserves careful engineering explanation.

A frozen airspeed signal at a value of, for example, 200 kt remains well within the absolute operational bounds of 60 to 400 kt defined in this thesis. Since the signal no longer changes after the fault onset, the sample-to-sample difference at every subsequent sample is determined entirely by the measurement noise applied before the freeze. Under low and medium noise conditions, these noise-induced differences are small and fall well below the jump threshold. The instantaneous change check therefore produces no violation, and the threshold detector generates no fault flag. The threshold method has no mechanism to recognize the absence of expected signal variation — it can only flag values that are too large or too extreme in absolute terms, not values that are suspiciously and persistently static [9].

This structural gap is not a tuning problem that can be resolved by adjusting the threshold values within the threshold-based detection framework. No matter how the absolute limits or jump thresholds are set, they cannot be made sensitive to the absence of variation without fundamentally changing the nature of the detection logic. Reducing the jump threshold to detect smaller differences would increase false alarm sensitivity to noise without addressing the fundamental issue. Expanding or reducing the absolute limits would change which absolute values are flagged but would still not detect a frozen signal that happens to be numerically within the acceptable range. The limitation is architectural, not parametric.

The partial detection observed under high noise conditions (miss rates of 78% in climb and 86% in descent) further illustrates this point. In those cases, the noise standard deviation of 2.0 kt occasionally produced sample-to-sample differences large enough to exceed the jump threshold even in the post-freeze segment. These coincidental detections are not genuine freeze recognitions. They are noise-induced false positives that happen to occur after the fault onset and are therefore counted as detections by the metric definition used in this thesis. Their occurrence is statistically unpredictable and operationally unreliable. A monitoring strategy that depends on measurement noise to detect a frozen signal would fail whenever the noise level decreases — for example, after sensor recalibration or in benign atmospheric conditions — and would therefore not provide any meaningful safety assurance. Relying on noise to detect a frozen signal is not an acceptable monitoring strategy in a certified avionics context [9], [14].

The rate-of-change method, by contrast, is structurally designed to identify the absence of signal variation. The freeze detection logic accumulates consecutive samples where the absolute rate of change falls below the minimum threshold of 0.2 kt/s and declares a fault flag only after this condition has been sustained for the predefined confirmation window of 5

seconds, equivalent to 50 consecutive samples at a sampling frequency of 10 Hz. This accumulation mechanism correctly identifies frozen behavior regardless of noise level because the frozen signal, by definition, produces a near-zero rate of change at every sample after the fault onset, and this near-zero rate persists continuously rather than occurring as an isolated event [9].

The consistent mean detection delay of approximately 5 seconds across all low and medium noise scenarios in both flight phases directly reflects this design. The delay is deterministic and governed entirely by the confirmation window length rather than by the noise level, the fault onset time, or the flight phase. This is confirmed visually by Picture 3.4, which shows extremely tight clustering of detection delay values at 5 seconds across all 50 Monte Carlo runs under low and medium noise conditions. The absence of run-to-run variability in detection delay demonstrates that the monitoring logic operates with near-perfect consistency under these conditions, which is a desirable property from a safety engineering perspective.

Under high noise conditions, the mean detection delay is slightly reduced to 3.9 seconds in the climb phase and 4.3 seconds in the descent phase. This reduction occurs because the larger noise-induced rate fluctuations in the post-freeze segment occasionally produce samples where the absolute rate of change exceeds the minimum threshold, briefly resetting the accumulation counter. However, because the overall trend of near-zero rate is maintained by the frozen signal, the counter recovers quickly and the confirmation condition is satisfied earlier on average than under low noise, where the counter increments monotonically from the fault onset. This noise-induced early detection is a secondary effect and does not impair the reliability of the detection, since zero missed detections are recorded under all noise conditions.

The 5-second confirmation window represents a deliberate design trade-off between detection speed and false alarm robustness. In flight-critical applications, this latency would need to be carefully evaluated against operational safety requirements. A frozen airspeed indication during a climb or descent phase could influence autopilot speed targeting, flight envelope protection activation, or crew decision-making within the 5-second interval before detection. Whether this delay is operationally acceptable depends on the specific aircraft system architecture, the rate of airspeed change during the affected phase, and the downstream consequences of acting on an incorrect airspeed value. These considerations reinforce the importance of formal threshold optimization in any certified implementation, as discussed in Section 1.7.

4.3. False Alarm Behavior and Phase Dependency

The identical false alarm counts produced by both detection methods across all scenarios require explicit explanation, since a naive reading of the results might suggest that the two methods are equivalent in all respects. During fault-free segments of the simulation, both detection methods apply the same instantaneous jump check to the noisy signal. This check is shared between the threshold and rate-of-change detection functions as implemented in the MATLAB code for this thesis. Any noise spike that produces a sample-to-sample difference exceeding the configured jump threshold will simultaneously trigger both detection flags, since both functions evaluate the same condition on the same input signal. This design overlap is the direct cause of the numerically identical false alarm statistics between the two methods in every scenario combination.

This result has an important practical implication. It demonstrates that under the current implementation, the rate-of-change method offers no improvement in false alarm performance compared to the threshold method during fault-free operation. The only differentiation between the two methods occurs during freeze fault scenarios, where the rate-of-change method correctly detects the frozen condition while the threshold method fails. For all other performance dimensions — bias detection, false alarm rate, and detection delay under non-freeze conditions — the two methods are functionally equivalent. Any future implementation aiming to reduce false alarm rates would therefore need to modify the shared jump check logic, since improving either method independently without addressing this shared component would not change the false alarm outcome.

The strong phase dependency observed in false alarm frequency is a consequence of the experimental design and the length of the fault-free observation window rather than any fundamental aerodynamic or physical difference between the climb and descent phases. In this thesis, false alarms are counted as the number of 0-to-1 transitions of the detection flag during the fault-free segment of each simulation run. The fault-free segment is defined as all samples preceding the fault onset time, which is randomized between 30% and 60% of the phase duration. For the descent phase, which spans from $t = 450$ seconds to $t = 600$ seconds, the maximum possible fault-free window is 150 seconds. For the climb phase, which spans from $t = 0$ to $t = 200$ seconds, the maximum fault-free window is 200 seconds in absolute terms, but the fault onset is constrained to the same 30% to 60% fractional range.

The substantially higher false alarm counts observed in the descent phase relative to the climb phase across all noise levels require further explanation beyond simple window length. The descent phase in the simulated profile is a linear decrease from 250 kt to 160 kt, producing a smooth and continuous rate of change. Under high noise conditions, the noise-induced sample-to-sample differences in this phase are superimposed on a non-zero underlying derivative, which means the combined signal derivative is more likely to produce occasional large excursions above the jump threshold than in the climb phase where the underlying derivative has a similar but slightly different characteristic. Additionally, the absolute airspeed values in the descent phase span a lower range, and the interaction between noise magnitude and the underlying signal dynamics differs subtly from the climb phase. The cumulative effect of these factors, combined with the length of the fault-free window, produces the observed descent-phase false alarm elevation.

Under high noise conditions, the mean false alarm count reaches approximately 577 events per run in descent-phase bias fault scenarios and 573 in descent-phase freeze fault scenarios. These values appear extreme in absolute terms and would be completely unacceptable in an operational avionics system. However, they must be interpreted correctly in context. The simulation runs at 10 Hz for a fault-free window that may extend to over 100 seconds, corresponding to more than 1000 samples. A false alarm rate of 577 events per run over this window implies that roughly half of all samples in the fault-free segment trigger the detection flag, which is consistent with the behavior of a jump threshold that is being exceeded by noise fluctuations at a high rate. This is not a pathological failure of the detection logic — it is the expected consequence of applying an unfiltered jump threshold with a fixed value to a signal whose noise magnitude is comparable to or exceeds the threshold sensitivity.

Real avionics monitoring systems address this problem through supplementary signal conditioning applied before the detection logic. Low-pass filtering or moving-average smoothing reduces the noise-induced variance of the discrete derivative, lowering the probability of threshold crossing without a genuine fault. Debouncing mechanisms require the detection flag to remain active for multiple consecutive samples before a fault condition is declared, suppressing single-sample noise triggers. Multi-sample confirmation windows, analogous to the freeze hold timer used in the rate-of-change detector, can be applied to jump detection as well, requiring sustained exceedance rather than a single-sample violation. The results of this thesis quantify the baseline false alarm behavior of completely unfiltered

classical methods and thereby identify precisely where such supplementary logic is necessary and what order-of-magnitude improvement it must deliver to achieve operationally acceptable performance [9], [10].

4.4. Comparative Assessment of Detection Methods

Considering the complete set of results, the two classical detection methods evaluated in this thesis exhibit a clearly defined pattern of complementary strengths and structural limitations that can be summarized along three performance dimensions: fault type coverage, detection delay, and false alarm robustness.

With respect to fault type coverage, the threshold-based method provides complete and immediate detection of constant bias faults but is structurally incapable of detecting frozen signal faults under realistic low and medium noise conditions. The rate-of-change method matches the threshold method identically for bias fault detection and additionally provides complete detection of frozen signal faults with zero missed detections across all tested scenarios. In terms of fault coverage breadth, the rate-of-change method is therefore strictly superior to the threshold method under the conditions evaluated in this thesis.

With respect to detection delay, both methods achieve negligible delay for bias faults. For freeze faults, the rate-of-change method introduces a fixed minimum delay governed by the confirmation window, which was set to 5 seconds in this implementation. This latency is an inherent property of the confirmation-based freeze detection mechanism and cannot be eliminated without compromising false alarm robustness. The threshold method, when it produces any detection at all for freeze faults — which occurs only incidentally under high noise conditions — does so with zero delay, but these detections are unreliable as discussed in Section 4.2. A fair comparison of detection delay must therefore account for the fact that the threshold method's apparent zero delay for freeze faults under high noise does not represent genuine detection capability.

With respect to false alarm robustness, both methods perform identically under all conditions tested, since they share the same jump check logic that drives false alarm generation. Neither method demonstrates any advantage over the other in terms of false alarm suppression. Under low noise conditions, both methods operate with negligible false alarm rates. Under medium noise, occasional false alarms appear, particularly in the descent phase. Under high

noise, both methods produce operationally unacceptable false alarm rates without supplementary filtering. This finding suggests that false alarm performance is independent of the choice between threshold and rate-of-change logic as implemented here and is instead determined entirely by the shared jump threshold and the noise characteristics of the signal.

This three-dimensional comparison is consistent with the established understanding in avionics fault detection literature that no single classical technique provides universal fault coverage across all fault types and operating conditions [9], [10]. The complementarity of threshold and rate-of-change methods observed in this thesis provides a concrete quantitative illustration of why certified avionics monitoring architectures implement multiple detection mechanisms in parallel. By combining threshold monitoring for abrupt deviation detection with rate-of-change monitoring for dynamic behavior anomalies, and supplementing both with cross-channel comparison and built-in test functions, real avionics systems achieve the layered fault coverage necessary to meet regulatory integrity requirements [2], [3], [14].

From a system design perspective, the results of this thesis suggest that for an airspeed monitoring system required to address both bias and freeze fault types, neither method should be implemented in isolation. A minimum effective architecture would combine the absolute limit check, the instantaneous jump check, and the freeze confirmation logic — with the addition of appropriate signal filtering to manage false alarm rates under high noise conditions. This combined architecture would leverage the detection strengths of both classical methods while mitigating their individual limitations. The formal optimization of the parameters of such a combined system — threshold values, filter characteristics, and confirmation window lengths — against specific aircraft operational requirements and regulatory safety objectives represents a natural next step beyond the present evaluation.

4.5. Limitations of the Study

Several limitations of the present study should be acknowledged. First, the analysis is based entirely on simulated data generated under simplified modeling assumptions. The airspeed profile is defined as a deterministic piecewise function without turbulence, atmospheric variability, or realistic aerodynamic coupling. While this controlled environment is appropriate for evaluating detection logic in isolation, it does not fully represent the complexity of real flight data. Second, only a single sensor channel is considered, excluding redundancy-based detection mechanisms that are standard in transport-category aircraft. Third,

the noise model assumes stationary Gaussian behavior, whereas real sensor noise may exhibit non-stationary characteristics, colored noise components, or intermittent disturbances. Fourth, only two fault types and two detection methods are evaluated, which limits the generalizability of the findings to the broader fault detection problem. Finally, the detection thresholds used in this thesis were selected based on physical reasoning and are not optimized for any specific aircraft type or certification standard. Threshold optimization against real operational data would be necessary before any of these methods could be applied in a certified avionics context.

4.6. Practical Implications and Engineering Recommendations

The findings of this thesis have several practical implications for the design and evaluation of airspeed monitoring systems in avionics engineering contexts. While the analysis is conducted in a simplified simulation environment, the structural conclusions regarding the capabilities and limitations of the two classical detection methods are directly applicable to real system design decisions.

The first and most fundamental implication is that fault type coverage must be explicitly verified for each monitoring method considered in a system design. The results demonstrate clearly that a method which is effective for one fault type may be completely ineffective for another, even when both fault types are physically plausible and operationally relevant. A monitoring architecture designed and validated only against bias faults would provide no protection against frozen signal conditions under realistic noise levels. This highlights the importance of defining a comprehensive fault catalog at the beginning of a monitoring system design effort and ensuring that each fault in the catalog is addressed by at least one detection mechanism with demonstrated effectiveness.

The second implication concerns the role of noise characterization in detection system design. The false alarm results show that the performance of both classical methods degrades significantly as noise magnitude increases. For any practical implementation, the expected noise characteristics of the sensor signal must be established through hardware testing and field data analysis before detection thresholds are selected. If the expected noise level is comparable to or exceeds the sensitivity required for fault detection, supplementary signal conditioning such as low-pass filtering or moving-average smoothing must be incorporated into the processing chain before the detection logic is applied. Selecting threshold parameters without knowledge of the actual sensor noise characteristics is likely to result in either excessive false alarms or insufficient detection sensitivity, depending on the direction of the error.

The third implication relates to the phase-dependent design of monitoring logic. The freeze detection mechanism implemented in this thesis suppresses the freeze check during cruise to avoid false alarms from the naturally stable cruise signal. This phase-awareness principle should be generalized to all detection logic applied to airspeed monitoring. The expected signal dynamics differ substantially between flight phases, and detection parameters that are appropriate during climb or descent may be inappropriate during cruise, takeoff, or approach. A robust monitoring architecture should therefore incorporate flight phase information — either from a dedicated flight phase identification function or from existing avionics data buses — to adapt detection sensitivity and threshold values to the current operational context.

The fourth implication concerns the 5-second detection latency introduced by the freeze confirmation window. For airspeed monitoring during dynamic flight phases such as climb and descent, a 5-second interval during which a frozen airspeed indication goes undetected may be operationally significant depending on the rate of airspeed change and the downstream consequences of acting on incorrect data. System designers implementing freeze detection logic should conduct a dedicated latency analysis that relates the maximum tolerable detection delay to the rate of divergence between the frozen value and the true airspeed, and to the operational impact of any system functions that rely on the airspeed input during the detection interval. The 5-second window used in this thesis is a reasonable starting point but should not be adopted without this analysis in a certified implementation.

Based on these observations, the following engineering recommendations can be derived from the results of this thesis. First, airspeed monitoring systems should implement both threshold-based and rate-of-change detection mechanisms in combination rather than relying on either method alone. Second, freeze detection logic should be restricted to dynamic flight phases and should incorporate a confirmation window whose length is formally justified against the maximum tolerable detection delay. Third, signal conditioning should be applied upstream of detection logic when operating under high noise conditions to maintain acceptable false alarm rates. Fourth, detection threshold parameters should be selected based on measured sensor noise characteristics rather than on physical reasoning alone. Fifth, the performance of any monitoring system should be validated through Monte Carlo simulation across the relevant operational envelope before deployment.

4.7. Directions for Future Research

The evaluation conducted in this thesis establishes a baseline characterization of classical fault detection performance for airspeed sensor monitoring. Several directions for future research follow directly from the findings and limitations identified in the preceding sections.

The most immediate extension of the present work is a systematic sensitivity analysis of bias fault detection as a function of fault magnitude. The current evaluation uses a fixed bias of +10 kt, which is large enough to trigger immediate detection by both methods under all tested conditions. Reducing the bias magnitude progressively would reveal the minimum detectable bias as a function of noise level and threshold parameters, providing a complete characterization of the detection sensitivity boundary. This analysis would quantify precisely the range of bias magnitudes that fall in the gap between detectable and undetectable, which is the operationally most important region for practical monitoring design.

A second direction involves the evaluation of gradual or ramp-type faults, in which the bias magnitude increases progressively over time rather than appearing as an instantaneous step. Gradual sensor drift is a physically realistic failure mode that classical threshold and rate-of-change methods may be particularly poorly suited to detect, since the rate of change of the signal remains within normal bounds throughout the fault evolution. Evaluating this fault type would test the limits of both classical methods in a qualitatively different way from the step-type faults considered in this thesis.

A third direction is the investigation of supplementary signal conditioning methods and their effect on the false alarm versus detection delay trade-off. Low-pass filtering, moving-average smoothing, and multi-sample confirmation logic each reduce false alarm rates at the cost of increased detection delay. Characterizing this trade-off quantitatively — for example, by plotting the receiver operating characteristic of the detection system as a function of filter parameters — would provide the design information needed to select conditioning parameters for a target operating point on the false alarm versus delay curve.

A fourth direction involves extending the evaluation framework to include residual-based monitoring using analytical redundancy. If an inertial reference model or an aerodynamic performance model is available to provide an independent airspeed estimate, the residual between the model prediction and the sensor measurement can be monitored for anomalies.

This approach has the potential to detect subtle biases that fall below the absolute threshold limits and to provide earlier detection of gradual faults than classical threshold logic. Comparing the performance of residual-based monitoring against the classical methods evaluated in this thesis, using the same simulation framework and performance metrics, would provide a direct and quantitative basis for assessing whether the additional complexity of model-based methods is justified by improved detection performance.

A fifth direction concerns the extension of the simulation framework to include more realistic signal models. The airspeed profile used in this thesis is a deterministic piecewise function without turbulence, atmospheric variability, or wind shear effects. Incorporating stochastic atmospheric disturbances modeled from flight data recorder measurements would increase the realism of the simulated signal and provide a more challenging test of detection robustness. Similarly, replacing the stationary Gaussian noise model with a more realistic non-stationary noise model that captures the frequency-dependent characteristics of real air data sensor noise would improve the fidelity of the false alarm analysis.

Finally, a sixth and longer-term direction is the application of the evaluation methodology developed in this thesis to real flight data. Access to recorded air data from operational aircraft — for example, through flight data monitoring programs operated by airlines or through publicly available accident investigation datasets — would allow the detection methods to be validated against genuine sensor behavior under real atmospheric conditions. This step would be necessary before any of the classical methods evaluated here could be considered for certified implementation and would provide the empirical grounding that complements the controlled analytical evaluation conducted in this thesis.

CONCLUSIONS

This thesis evaluated the effectiveness of classical fault detection methods applied to aircraft airspeed sensor data using a simulation-based methodology. Two fault types — constant bias and frozen signal — were modeled and injected into simulated airspeed profiles representing climb and descent flight phases. Two detection methods — threshold-based monitoring and rate-of-change monitoring — were implemented in MATLAB and evaluated through a Monte Carlo campaign of 600 simulation runs across twelve scenario combinations of fault type, noise level, and flight phase. Performance was assessed using three metrics: mean detection delay, missed detection rate, and mean false alarm event count per run.

The following conclusions are drawn from the results of the simulation campaign and the subsequent engineering analysis.

First, threshold-based monitoring is highly effective for detecting constant bias faults of the magnitude evaluated in this thesis. Both the threshold-based and rate-of-change detectors achieved a missed detection rate of zero across all noise levels and flight phases, with negligible mean detection delay. This result confirms that an abrupt step-type bias of +10 kt is reliably captured by instantaneous jump threshold logic regardless of measurement noise conditions.

Second, threshold-based monitoring is structurally incapable of detecting frozen signal faults under realistic low and medium noise conditions. The missed detection rate for this method reached 100% under low and medium noise in both flight phases, confirming that the absence of signal variation is invisible to absolute-limit and jump-threshold logic. Partial detection observed under high noise conditions was caused by noise-induced threshold crossings rather than genuine freeze recognition and cannot be considered a reliable detection capability.

Third, rate-of-change monitoring with a confirmation window successfully detected all frozen signal faults across all tested noise levels and flight phases, achieving a missed detection rate of zero in every scenario. The mean detection delay of approximately 5 seconds under low and medium noise conditions is a direct consequence of the confirmation window length and is consistent and predictable across Monte Carlo runs. This deterministic latency is an inherent property of the confirmation-based detection mechanism and represents a deliberate engineering trade-off between detection speed and false alarm robustness.

Fourth, both detection methods produced identical false alarm counts in every scenario. This is a consequence of the shared instantaneous jump check logic implemented in both detection functions. False alarm frequency increased substantially with noise magnitude and was consistently higher in descent-phase scenarios than in climb-phase scenarios due to the longer fault-free observation window available during descent. Under high noise conditions, false alarm rates reached operationally unacceptable levels for both methods, indicating that supplementary signal conditioning is necessary before either method can be applied in a high-noise environment.

Fifth, neither method alone provides complete fault coverage across both fault types evaluated. Threshold-based monitoring is appropriate for large abrupt deviations but fails for frozen signals. Rate-of-change monitoring addresses both fault types but introduces a fixed detection latency for freeze faults and shares the false alarm sensitivity of the threshold method. These findings quantitatively confirm the established principle in avionics fault detection that no single classical technique provides universal coverage and that layered monitoring architectures combining multiple complementary methods are necessary for comprehensive fault detection.

Sixth, the regulatory and certification context reviewed in Chapter 1 establishes that any practical deployment of classical detection methods in certified avionics software must satisfy requirements beyond those addressed in this thesis, including formal threshold optimization, software development assurance, and environmental qualification. The results of this thesis provide a quantitative analytical foundation for such development but do not constitute a demonstration of regulatory compliance.

The research objective stated in the Introduction — to evaluate the effectiveness of classical fault detection methods for detecting constant bias and frozen signal faults using simulated flight data — has been fully achieved. The results provide a structured, reproducible, and quantitatively grounded comparison of threshold-based and rate-of-change monitoring under controlled conditions, clearly identifying the conditions under which each method succeeds or fails.

Directions for future research identified in Section 4.7 include systematic variation of bias magnitude to characterize detection sensitivity boundaries, evaluation of gradual ramp-type faults, investigation of supplementary signal conditioning methods, extension to residual-

based and model-based detection, incorporation of more realistic atmospheric and noise models, and validation against real flight data. These extensions would progressively close the gap between the controlled simulation evaluation conducted in this thesis and the conditions encountered in certified operational systems.

BIBLIOGRAPHY

- [1] Collinson, R. P. G. *Introduction to Avionics Systems*. – 3rd ed. – Dordrecht: Springer, 2011. – 530 pp.
- [2] Moir, I., Seabridge, A. *Aircraft Systems: Mechanical, Electrical and Avionics Subsystems Integration*. – 3rd ed. – Chichester: Wiley, 2008. – 504 pp.
- [3] Spitzer, C. R. *Digital Avionics Systems: Principles and Practice*. – 2nd ed. – Boca Raton: CRC Press, 2001. – 448 pp.
- [4] Lu, P., Van Eykeren, L., Van Kampen, E., Chu, Q. P. Air Data Sensor Fault Detection and Diagnosis with Application to Real Flight Data // *AIAA Guidance, Navigation, and Control Conference*. – Kissimmee, Florida, USA, 2015. – AIAA 2015-1049.
- [5] Castaldi, P., Mimmo, N., Simani, S. Avionic Air Data Sensors Fault Detection and Isolation by Singular Perturbation // *Sensors*. – 2017. – Vol. 17, No. 10. – pp. 1–22.
- [6] Lima Lopes, L. et al. A Review of Fault Diagnosis Techniques Applied to Aircraft Air Data Systems // *OASICS – Dagstuhl OpenAccess Series in Informatics*. – 2024.
- [7] Federal Aviation Administration. *Pilot's Handbook of Aeronautical Knowledge*. – FAA-H-8083-25B. – Washington, D.C.: FAA, 2016.
- [8] Federal Aviation Administration. *Airplane Flying Handbook*. – FAA-H-8083-3B. – Washington, D.C.: FAA, 2016.
- [9] Isermann, R. *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. – Berlin: Springer, 2006. – 475 pp.
- [10] Blanke, M., Kinnaert, M., Lunze, J., Staroswiecki, M. *Diagnosis and Fault-Tolerant Control*. – 2nd ed. – Berlin: Springer, 2006. – 672 pp.
- [11] Freeman, P., Seiler, P., Balas, G. J. Air Data System Fault Modeling and Detection // *Control Engineering Practice*. – 2013. – Vol. 21, No. 10. – pp. 1290–1303.
- [12] Stevens, B. L., Lewis, F. L. *Aircraft Control and Simulation*. – 3rd ed. – Hoboken: Wiley, 2015. – 768 pp.
- [13] Bureau d'Enquêtes et d'Analyses pour la Sécurité de l'Aviation Civile (BEA). *Final Report: Accident on 1st June 2009 to the Airbus A330-203 Registered F-GZCP Operated by Air France Flight AF 447 Rio de Janeiro – Paris*. – Paris: BEA, 2012.
- [14] RTCA Inc. *Environmental Conditions and Test Procedures for Airborne Equipment*. – RTCA DO-160G. – Washington, D.C.: RTCA Inc., 2010.
- [15] Patton, R. J., Frank, P. M., Clark, R. N. *Issues of Fault Diagnosis for Dynamic Systems*. – London: Springer, 2000. – 645 pp.

[16] Hajiyeve, C., Caliskan, F. *Fault Diagnosis and Reconfiguration in Flight Control Systems*. – Boston: Springer, 2003. – 268 pp.

[17] European Union Aviation Safety Agency (EASA). *Certification Specifications and Acceptable Means of Compliance for Large Aeroplanes CS-25*. – Amendment 27. – Cologne: EASA, 2023.

[18] Robert, C. P., Casella, G. *Monte Carlo Statistical Methods*. – 2nd ed. – New York: Springer, 2004. – 645 pp.

APPENDICES

APPENDIX NO. 1

SIMULATION CONFIGURATION PARAMETERS

File: config_default.m

Central configuration function defining all simulation parameters including time base, airspeed profile values, noise levels, fault settings, detection thresholds, and output directory paths. This function is called at the start of every simulation run to ensure consistent parameter values across all scenarios.

Source Code:

```
function cfg = config_default()

%CONFIG_DEFAULT Central configuration for simulations and detection.

% --- Time base ---

cfg.fs = 10;                % Hz
cfg.dt = 1/cfg.fs;          % s
cfg.T_total = 600;          % total duration (s)

% Phase split (seconds) - piecewise profile
cfg.t1 = 200;                % end of climb
cfg.t2 = 450;                % end of cruise (start of descent)

% --- Airspeed profile (IAS/CAS direct) ---
cfg.V0 = 140;                % kt
cfg.Vc = 250;                % kt
cfg.Vf = 160;                % kt

% Optional small cruise variability
cfg.cruise_sine_amp = 1.0;  % kt
```

```

cfg.cruise_sine_freq = 0.02; % Hz

% --- Noise levels (kt, std dev) ---
cfg.noise_levels = struct( ...
    "low", 0.5, ...
    "med", 1.0, ...
    "high", 2.0);

% --- Fault settings ---
cfg.bias_mag_kt = 10;          % +10 kt bias
cfg.freeze_min_rate = 0.2;    % kt/s threshold for "nearly zero"
cfg.freeze_hold_s = 5.0;      % seconds needed to declare freeze

% Fault onset window as fraction of phase duration
cfg.fault_onset_frac_lo = 0.30;
cfg.fault_onset_frac_hi = 0.60;

% --- Detection thresholds ---
cfg.Vmin = 60;                % kt
cfg.Vmax = 400;               % kt
cfg.jump_max_kt_per_s = 40;   % kt/s
cfg.rate_max_kt_per_s = 40;   % kt/s

% --- Monte Carlo settings ---
cfg.N_runs_per_scenario = 50;

% --- Output ---
cfg.out_dir = fullfile(pwd, "output");
cfg.out_csv_dir = fullfile(cfg.out_dir, "runs_csv");
cfg.out_fig_dir = fullfile(cfg.out_dir, "figures");
end

```

APPENDIX NO. 2

AIRSPPEED PROFILE GENERATION

File: generate_profile.m

Generates the piecewise clean airspeed profile representing the three flight phases: climb (linear increase from V_0 to V_c), cruise (constant with small sinusoidal perturbation), and descent (linear decrease from V_c to V_f). Returns the time vector, clean airspeed vector, and phase label array.

Source Code:

```
function [t, V_clean, phase] = generate_profile(cfg)
%GENERATE_PROFILE Piecewise IAS profile: climb -> cruise -> descent.

t = (0:cfg.dt:cfg.T_total).';
n = numel(t);

V_clean = zeros(n,1);
phase = strings(n,1);

% Phase indices
i1 = t < cfg.t1;
i2 = (t >= cfg.t1) & (t < cfg.t2);
i3 = t >= cfg.t2;

% Climb: linear increase V0 -> Vc
V_clean(i1) = cfg.V0 + (cfg.Vc - cfg.V0) * (t(i1) / cfg.t1);
phase(i1) = "climb";
```

```

% Cruise: constant + small sine
tau = t(i2) - cfg.t1;
V_clean(i2) = cfg.Vc + cfg.cruise_sine_amp * ...
    sin(2*pi*cfg.cruise_sine_freq * tau);
phase(i2) = "cruise";

% Descent: linear decrease Vc -> Vf
t_des = t(i3) - cfg.t2;
T_des = cfg.T_total - cfg.t2;
V_clean(i3) = cfg.Vc + (cfg.Vf - cfg.Vc) * (t_des / T_des);
phase(i3) = "descent";
end

```

APPENDIX NO. 3

MEASUREMENT NOISE INJECTION

File: add_noise.m

Applies additive white Gaussian noise to the clean airspeed signal. The noise standard deviation σ corresponds to the noise level selected for each scenario (low = 0.5 kt, medium = 1.0 kt, high = 2.0 kt). Noise is applied prior to fault injection to replicate realistic sensor measurement conditions.

Source Code:

```
function V_noisy = add_noise(V_clean, sigma)
%ADD_NOISE Additive Gaussian noise.
%
%   V_noisy = add_noise(V_clean, sigma)
%
%   Inputs:
%       V_clean : clean airspeed vector (kt)
%       sigma   : noise standard deviation (kt)
%
%   Output:
%       V_noisy : noisy airspeed vector (kt)

V_noisy = V_clean + sigma * randn(size(V_clean));
end
```

APPENDIX NO. 4

CONSTANT BIAS FAULT INJECTION

File: inject_bias.m

Injects a constant bias fault by adding a fixed offset (bias_mag) to the airspeed signal at all time samples from t_fault onward. Returns the faulty signal and a logical fault mask indicating which samples are affected. The bias magnitude used in the simulation campaign is +10 kt.

Source Code:

```
function [V_faulty, fault_mask] = inject_bias(t, V_in, t_fault,
bias_mag)

%INJECT_BIAS Adds constant bias after t_fault.

% [V_faulty, fault_mask] = inject_bias(t, V_in, t_fault, bias_mag)
%
% Inputs:
% t : time vector (s)
% V_in : input airspeed vector (kt)
% t_fault : fault onset time (s)
% bias_mag : bias magnitude (kt), positive = upward shift
%
% Outputs:
% V_faulty : airspeed with bias applied after t_fault (kt)
% fault_mask : logical array, true where fault is active

fault_mask = t >= t_fault;
V_faulty = V_in;
V_faulty(fault_mask) = V_in(fault_mask) + bias_mag;
end
```

APPENDIX NO. 5

FROZEN SIGNAL FAULT INJECTION

File: inject_freeze.m

Injects a frozen signal fault by locking the airspeed output at the value recorded at the fault onset time t_{fault} . All subsequent samples are held at this constant value, simulating a stuck pitot tube or signal processing failure that causes the sensor output to stop updating.

Source Code:

```
function [V_faulty, fault_mask] = inject_freeze(t, V_in, t_fault)
%INJECT_FREEZE Freezes signal at value at t_fault.
%
% [V_faulty, fault_mask] = inject_freeze(t, V_in, t_fault)
%
% Inputs:
%   t           : time vector (s)
%   V_in        : input airspeed vector (kt)
%   t_fault     : fault onset time (s)
%
% Outputs:
%   V_faulty    : airspeed frozen at value V_in(t_fault) (kt)
%   fault_mask  : logical array, true where fault is active

fault_mask = t >= t_fault;
V_faulty = V_in;

idx0 = find(t >= t_fault, 1, "first");
if isempty(idx0)
```

```
        return;  
    end  
    V0 = V_in(idx0);  
    V_faulty(fault_mask) = V0;  
end
```

APPENDIX NO. 6

THRESHOLD-BASED FAULT DETECTION

File: detect_threshold.m

Implements threshold-based fault detection. Two conditions are monitored: (1) absolute airspeed limits — a fault flag is raised if the signal falls below V_{min} or exceeds V_{max} ; (2) instantaneous jump limit — a fault flag is raised if the sample-to-sample change exceeds the configured maximum jump rate. The output is a binary flag vector F where 1 indicates a detected fault condition.

Source Code:

```
function F = detect_threshold(t, V, cfg)

%DETECT_THRESHOLD Absolute limits + jump limit.

%
%   F = detect_threshold(t, V, cfg)
%
%   Inputs:
%       t    : time vector (s)
%       V    : airspeed signal to be monitored (kt)
%       cfg  : configuration struct from config_default()
%
%   Output:
%       F    : binary fault flag vector (1 = fault detected, 0 = normal)
%
%   Detection logic:
%       (1) Absolute limits:  V < Vmin  OR  V > Vmax
%       (2) Jump limit:      |dV| > jump_max_kt_per_s * dt

F = zeros(size(V));
```

```

% Absolute bounds
abs_bad = (V < cfg.Vmin) | (V > cfg.Vmax);

% Jump limit (convert kt/s to per-sample magnitude)
jump_limit = cfg.jump_max_kt_per_s * cfg.dt;
dV = [0; abs(diff(V))];
jump_bad = dV > jump_limit;

F(abs_bad | jump_bad) = 1;
end

```

APPENDIX NO. 7

RATE-OF-CHANGE FAULT DETECTION

File: detect_rate.m

Implements rate-of-change fault detection with phase-awareness. Two conditions are monitored: (1) excessive rate — a fault flag is raised if the magnitude of the discrete derivative exceeds the configured maximum rate; (2) frozen signal detection — a fault flag is raised if the rate of change remains below a minimum threshold for a continuous duration exceeding the configured freeze hold time. Frozen detection is suppressed during cruise phase to avoid false alarms from the naturally stable cruise signal.

Source Code:

```
function F = detect_rate(t, V, phase, cfg)

%DETECT_RATE Excessive rate + frozen detection (phase-aware).

%

% F = detect_rate(t, V, phase, cfg)

%

% Inputs:

% t      : time vector (s)

% V      : airspeed signal to be monitored (kt)

% phase  : string array of flight phase labels per sample

% cfg    : configuration struct from config_default()

%

% Output:

% F      : binary fault flag vector (1 = fault detected, 0 = normal)

%

% Detection logic:

% (1) Excessive rate: |dV/dt| > rate_max_kt_per_s
```

```

%      (2) Frozen signal:   |dV/dt| < freeze min rate for >=
freeze_hold_s

%      (only evaluated during climb and descent phases)

F = zeros(size(V));

% Discrete rate of change (kt/s)
dVdt = [0; diff(V)] ./ cfg.dt;

% Condition 1: Excessive rate
rate_bad = abs(dVdt) > cfg.rate_max_kt_per_s;

% Condition 2: Frozen detection
% Only meaningful in climb/descent – suppress during cruise
active_phase = (phase == "climb") | (phase == "descent");
near_zero = abs(dVdt) < cfg.freeze_min_rate;
near_zero = near_zero & active_phase;

% Require near-zero rate for a continuous duration >= freeze_hold_s
N_hold = max(1, round(cfg.freeze_hold_s / cfg.dt));
frozen_bad = false(size(V));
count = 0;
for k = 1:numel(V)
    if near_zero(k)
        count = count + 1;
    else
        count = 0;
    end
    if count >= N_hold
        frozen_bad(k) = true;
    end
end

```

```
end
```

```
F(rate_bad | frozen_bad) = 1;
```

```
end
```

APPENDIX NO. 8

DETECTION PERFORMANCE METRIC COMPUTATION

File: compute_metrics.m

Computes the three performance metrics for each simulation run: detection delay (time between fault onset and first detection), missed detection indicator (1 if the fault was never detected within the observation window), and false alarm count (number of detection flag transitions during fault-free segments). These metrics are aggregated across Monte Carlo runs to produce the scenario-level statistics reported in Chapter 3.

Source Code:

```
function m = compute_metrics(t, fault_mask, F)
%COMPUTE_METRICS Detection delay, missed detection, false alarm count.
%
% m = compute_metrics(t, fault_mask, F)
%
% Inputs:
% t : time vector (s)
% fault_mask : logical array, true where fault is active
% F : detector flag vector (0 = normal, 1 = fault detected)
%
% Output:
% m : struct with fields:
% m.delay s - detection delay in seconds (NaN if missed)
% m.missed - 1 if fault not detected, 0 otherwise
% m.false_alarms - number of false alarm events
```

```

m = struct();
m.missed = 0;
m.delay_s = NaN;
m.false_alarms = 0;

% False alarms: flag transitions during fault-free segment
F_nf = F(~fault_mask) > 0;
m.false_alarms = sum(diff([0; F_nf]) == 1);

% Find first sample where fault is active
idx_fault_start = find(fault_mask, 1, "first");
if isempty(idx_fault_start)
    return; % No fault in this run (not used in current scope)
end

% Find first detection at or after fault onset
idx_detect = find(((1:numel(F))' >= idx_fault_start) & (F > 0), ...
    1, "first");

if isempty(idx_detect)
    m.missed = 1; % Fault present but never detected
    return;
end

m.delay_s = t(idx_detect) - t(idx_fault_start);
end

```

APPENDIX NO. 9

MONTE CARLO BATCH SIMULATION RUNNER

File: main_run_batch.m

Main script executing the full Monte Carlo simulation campaign. Iterates over all combinations of fault type (bias, freeze), noise level (low, medium, high), and flight phase (climb, descent), performing 50 randomised runs per scenario. For each run, a unique deterministic seed ensures reproducibility. Results are saved as individual CSV files and aggregated into a summary table. This script produced the complete dataset of 600 simulation runs reported in Chapter 3.

Source Code:

```
% main_run_batch.m

% Full Monte Carlo simulation campaign.

% Iterates over fault types, noise levels, and flight phases.

% Saves per-run CSV files and an aggregated summary table.

%

% Total runs: 2 fault types x 3 noise levels x 2 phases x 50 runs = 600

clear; clc;

cfg = config_default();

fault_types    = ["bias", "freeze"];
noise_names    = ["low", "med", "high"];
phase_focuses  = ["climb", "descent"];

all_rows = [];
run_counter = 0;
```

```

for ft = fault_types
    for nn = noise_names
        for pf = phase_focuses

            sigma = cfg.noise_levels.(nn);

            for r = 1:cfg.N_runs_per_scenario

                run_counter = run_counter + 1;
                seed = run_counter;    % deterministic, unique per run

                utils.set_seed(seed);

                % --- Generate profile and add noise
                [t, V_clean, phase] = sim.generate_profile(cfg);
                V_noisy = sim.add_noise(V_clean, sigma);

                % --- Randomise fault onset within phase window
                if pf == "climb"
                    t_start = 0;        t_end = cfg.t1;
                else
                    t_start = cfg.t2;    t_end = cfg.T_total;
                end
                frac = cfg.fault_onset_frac_lo + ...
                    (cfg.fault_onset_frac_hi - cfg.fault_onset_frac_lo) *
rand;

                tf = t_start + frac * (t_end - t_start);

                % --- Inject fault
                if ft == "bias"
                    [V_faulty, fault_mask] = faults.inject_bias( ...

```

```

        t, V_noisy, tf, cfg.bias_mag_kt);
else
    [V_faulty, fault_mask] = faults.inject_freeze( ...
        t, V_noisy, tf);
end

% --- Run detectors
F_T = detect.detect_threshold(t, V_faulty, cfg);
F_R = detect.detect_rate(t, V_faulty, phase, cfg);

% --- Compute metrics
mT = eval.compute_metrics(t, fault_mask, F_T);
mR = eval.compute_metrics(t, fault_mask, F_R);

% --- Save per-run CSV and metadata
run_id = utils.make_run_id(ft, nn, seed, pf);
meta    = struct("fault_type", ft, "noise", nn, ...
                "seed", seed, "phase_focus", pf, ...
                "t_fault", tf);
utils.save_run_csv(cfg, run_id, t, phase, ...
    V_clean, V_noisy, V_faulty, fault_mask, F_T, F_R, meta);

% --- Append summary row
row = table(string(run_id), string(ft), string(nn), ...
    string(pf), tf, ...
    mT.delay_s, mT.missed, mT.false_alarms, ...
    mR.delay_s, mR.missed, mR.false_alarms, ...
    'VariableNames', [ ...
        "run_id", "fault_type", "noise", "phase_focus", "t_fault",
...
        "T_delay_s", "T_missed", "T_false_alarms", ...

```

```

        "R_delay_s", "R_missed", "R_false_alarms"]);

    all_rows = [all_rows; row]; %#ok<AGROW>

end % runs

end % phase

end % noise

end % fault type

% --- Save aggregated summary
if ~exist(cfg.out_dir, "dir"), mkdir(cfg.out_dir); end
summary_path = fullfile(cfg.out_dir, "summaries", "summary_runs.csv");
if ~exist(fileparts(summary_path), "dir")
    mkdir(fileparts(summary_path));
end
writetable(all_rows, summary_path);

disp("Batch complete. Summary saved to:");
disp(summary_path);

```

APPENDIX NO. 10

STATISTICAL POST-PROCESSING OF SIMULATION RESULTS

File: analyze_results.py

Language: Python 3 | Dependencies: pandas, numpy, matplotlib

Loads the Monte Carlo simulation output file `summary_runs.csv` produced by `main_run_batch.m` (Appendix No. 9) and computes grouped performance metrics for each scenario combination of fault type, noise level, and flight phase. Detection delay is averaged only over non-missed runs, consistent with the methodology defined in Section 2.7.1. The script exports `summary_grouped.csv` and three formatted CSV tables corresponding to Tables 3.1, 3.2, and 3.3 in Chapter 3. All grouped statistics reported in the Results chapter were produced by this script.

Source Code:

```
"""
analyze_results.py
-----

Statistical post-processing of Monte Carlo simulation results.

Loads summary_runs.csv produced by main_run_batch.m, computes grouped
performance metrics, and exports:

- summary_grouped.csv      : mean metrics per scenario
- table_3_1_bias.csv       : Chapter 3, Table 3.1 (bias fault results)
- table_3_2_freeze.csv     : Chapter 3, Table 3.2 (freeze fault results)
- table_3_3_fa.csv         : Chapter 3, Table 3.3 (false alarm analysis)

Usage:

python analyze_results.py
```

Input:

```
output/summaries/summary_runs.csv
```

Outputs:

```
output/summaries/summary_grouped.csv
```

```
output/summaries/table_3_1_bias.csv
```

```
output/summaries/table_3_2_freeze.csv
```

```
output/summaries/table_3_3_fa.csv
```

Dependencies:

```
pandas >= 1.3
```

```
numpy >= 1.21
```

```
"""
```

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
# — Paths —————
```

```
INPUT_PATH = os.path.join("output", "summaries", "summary_runs.csv")
```

```
OUTPUT_DIR = os.path.join("output", "summaries")
```

```
NOISE_ORDER = ["low", "med", "high"]
```

```
NOISE_LABEL = {"low": "Low", "med": "Medium", "high": "High"}
```

```
PHASE_ORDER = ["climb", "descent"]
```

```
PHASE_LABEL = {"climb": "Climb", "descent": "Descent"}
```

```
def load_data(path):
```

```
    """Load and validate the simulation run summary CSV."""
```

```
    if not os.path.exists(path):
```

```

        raise FileNotFoundError(

            f"Input file not found: {path}\n"

            "Run main run batch.m in MATLAB first to generate simulation
results."

        )

df = pd.read_csv(path)

required = [

    "fault_type", "noise", "phase_focus",

    "T_delay_s", "T_missed", "T_false_alarms",

    "R_delay_s", "R_missed", "R_false_alarms"

]

missing = [c for c in required if c not in df.columns]

if missing:

    raise ValueError(f"Missing columns in input CSV: {missing}")

print(f"Loaded {len(df)} simulation runs from: {path}")

return df


def compute_grouped_metrics(df):

    """

    Compute mean detection delay, missed detection rate, and mean false
    alarm count per scenario group (fault_type x noise x phase_focus).

    Detection delay is averaged only over non-missed runs (missed == 0),
    consistent with the thesis methodology Section 2.7.1.

    """

    rows = []

    groups = df.groupby(["fault_type", "noise", "phase_focus"])

    for (ft, nn, pf), grp in groups:

        n = len(grp)

        # Threshold detector metrics

```

```

    T_detected = grp[grp["T_missed"] == 0]

    T_delay_mean = T_detected["T_delay_s"].mean() if len(T_detected) > 0
else np.nan

    T_miss_rate = grp["T_missed"].mean()

    T_FA_mean = grp["T_false_alarms"].mean()

# Rate-of-change detector metrics

    R_detected = grp[grp["R_missed"] == 0]

    R_delay_mean = R_detected["R_delay_s"].mean() if len(R_detected) > 0
else np.nan

    R_miss_rate = grp["R_missed"].mean()

    R_FA_mean = grp["R_false_alarms"].mean()

rows.append({

    "fault_type":    ft,

    "noise":         nn,

    "phase_focus":   pf,

    "T_delay_mean":  round(T_delay_mean, 4) if not
np.isnan(T_delay_mean) else np.nan,

    "T_miss_rate":   round(T_miss_rate, 4),

    "T_FA_mean":     round(T_FA_mean, 4),

    "R_delay_mean":  round(R_delay_mean, 4) if not
np.isnan(R_delay_mean) else np.nan,

    "R_miss_rate":   round(R_miss_rate, 4),

    "R_FA_mean":     round(R_FA_mean, 4),

    "n":             n

})

grouped = pd.DataFrame(rows)

# Sort by fault_type, noise level order, phase
grouped["noise_order"] = grouped["noise"].map(

    {k: i for i, k in enumerate(NOISE_ORDER)}

)

```

```

grouped = grouped.sort_values(
    ["fault_type", "noise_order", "phase_focus"]
).drop(columns=["noise_order"]).reset_index(drop=True)

return grouped

def format_delay(val):
    """Format detection delay for thesis tables. NaN -> em dash."""
    if pd.isna(val):
        return "-"
    return f"{val:.2f}"

def build_table_3_1_bias(grouped):
    """
    Build Table 3.1: Mean Detection Metrics for Bias Faults.

    Columns: Noise | Phase | T Delay (s) | T Miss Rate | R Delay (s) | R Miss
    Rate
    """
    bias = grouped[grouped["fault_type"] == "bias"].copy()
    rows = []
    for _, row in bias.iterrows():
        rows.append({
            "Noise": NOISE_LABEL.get(row["noise"], row["noise"]),
            "Phase": PHASE_LABEL.get(row["phase focus"],
row["phase_focus"]),
            "T Delay (s)": format_delay(row["T_delay_mean"]),
            "T Miss Rate": f"{row['T_miss_rate']:.2f}",
            "R Delay (s)": format_delay(row["R_delay_mean"]),
            "R Miss Rate": f"{row['R_miss_rate']:.2f}",
        })
    return pd.DataFrame(rows)

```

```

def build_table_3_2_freeze(grouped):
    """
    Build Table 3.2: Mean Detection Metrics for Freeze Faults.

    Columns: Noise | Phase | T Delay (s) | T Miss Rate | R Delay (s) | R Miss
    Rate
    """
    freeze = grouped[grouped["fault_type"] == "freeze"].copy()
    rows = []
    for _, row in freeze.iterrows():
        rows.append({
            "Noise": NOISE_LABEL.get(row["noise"], row["noise"]),
            "Phase": PHASE_LABEL.get(row["phase focus"],
row["phase_focus"]),
            "T Delay (s)": format_delay(row["T_delay_mean"]),
            "T Miss Rate": f"{row['T_miss_rate']:.2f}",
            "R Delay (s)": format_delay(row["R_delay_mean"]),
            "R Miss Rate": f"{row['R_miss_rate']:.2f}",
        })
    return pd.DataFrame(rows)

def build_table_3_3_false_alarms(grouped):
    """
    Build Table 3.3: Mean False Alarm Events per Run.

    Columns: Fault | Noise | Phase | T FA Mean | R FA Mean
    """
    rows = []
    for _, row in grouped.iterrows():
        rows.append({
            "Fault": row["fault_type"].capitalize(),
            "Noise": NOISE_LABEL.get(row["noise"], row["noise"]),

```

```

        "Phase":      PHASE_LABEL.get(row["phase focus"],
row["phase_focus"]),

        "T FA Mean": f"{row['T_FA_mean']:.2f}",

        "R FA Mean": f"{row['R_FA_mean']:.2f}",

    })

    return pd.DataFrame(rows)

def print_table(title, df):

    """Print a formatted table to console."""

    print(f"\n{'='*70}")

    print(f"    {title}")

    print('='*70)

    print(df.to_string(index=False))

def main():

    os.makedirs(OUTPUT_DIR, exist_ok=True)

    # Load data

    df = load_data(INPUT_PATH)

    # Compute grouped metrics

    print("\nComputing grouped scenario metrics...")

    grouped = compute_grouped_metrics(df)

    # Save grouped summary

    grouped_path = os.path.join(OUTPUT_DIR, "summary_grouped.csv")

    grouped.to_csv(grouped_path, index=False)

    print(f"Saved: {grouped_path}")

    # Build and save thesis tables

    t31 = build_table_3_1_bias(grouped)

```

```

t32 = build_table_3_2_freeze(grouped)

t33 = build_table_3_3_false_alarms(grouped)


t31.to_csv(os.path.join(OUTPUT_DIR, "table_3_1_bias.csv"), index=False)
t32.to_csv(os.path.join(OUTPUT_DIR, "table_3_2_freeze.csv"), index=False)
t33.to_csv(os.path.join(OUTPUT_DIR, "table_3_3_fa.csv"), index=False)
print("Saved: table_3_1_bias.csv, table_3_2_freeze.csv, table_3_3_fa.csv")


# Print tables to console for verification
print_table("Table 3.1 – Bias Fault Detection Metrics", t31)
print_table("Table 3.2 – Freeze Fault Detection Metrics", t32)
print_table("Table 3.3 – False Alarm Analysis", t33)


# Summary statistics
print(f"\n{'='*70}")

print("  CAMPAIGN SUMMARY")

print('='*70)

print(f"  Total simulation runs   : {len(df)}")

print(f"  Fault types             : {sorted(df['fault_type'].unique())}")

print(f"  Noise levels            : {NOISE_ORDER}")

print(f"  Flight phases           : {sorted(df['phase_focus'].unique())}")

print(f"  Runs per scenario       : {df.groupby(['fault_type', 'noise', 'phase_focus']).size().iloc[0]}")

print(f"  Scenarios total         : {len(grouped)}")

print(f"\nPost-processing complete.")


if __name__ == "__main__":
    main()

```

APPENDIX NO. 11

RESULTS VISUALISATION

File: plot_results.py

Language: Python 3 | Dependencies: pandas, numpy, matplotlib

Generates four publication-quality figures from the grouped simulation results. Figure A (Pic. 3.1) shows mean detection delay for bias faults across noise levels and flight phases for both detectors. Figure B (Pic. 3.2.) shows missed detection rates for freeze faults. Figure C (Pic. 3.3.) shows mean false alarm event counts per run across all scenario combinations. Figure D (Pic. 3.4.) presents box plot distributions of detection delay for freeze faults under the rate-of-change detector, illustrating the spread and consistency of the 5-second freeze confirmation window across Monte Carlo runs. All figures are exported as 300 DPI PNG files for inclusion in the thesis.

Source Code:

```
"""
plot_results.py
-----

Results visualisation for Monte Carlo simulation campaign.

Loads summary_grouped.csv produced by analyze_results.py and generates
four publication-quality figures:

Figure 1 – Detection delay comparison (bias faults, both detectors)
Figure 2 – Missed detection rate (freeze faults, both detectors)
Figure 3 – False alarm count vs noise level (grouped bar chart)
Figure 4 – Detection delay distribution (freeze faults, rate detector,
          box plots across noise levels and phases)
```

Usage:

```
python plot_results.py
```

Input:

```
output/summaries/summary_grouped.csv
```

```
output/summaries/summary_runs.csv
```

Outputs:

```
output/figures/py_fig_A_bias_delay.png
```

```
output/figures/py_fig_B_freeze_miss.png
```

```
output/figures/py_fig_C_false_alarms.png
```

```
output/figures/py_fig_D_freeze_delay_dist.png
```

Dependencies:

```
pandas      >= 1.3
```

```
numpy       >= 1.21
```

```
matplotlib >= 3.4
```

```
"""
```

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib
```

```
matplotlib.use("Agg") # non-interactive backend for script use
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.ticker as ticker
```

```
# — Paths —————
```

```
GROUPED_PATH = os.path.join("output", "summaries", "summary_grouped.csv")
```

```
RUNS_PATH    = os.path.join("output", "summaries", "summary_runs.csv")
```

```
FIG_DIR      = os.path.join("output", "figures")
```

```

# — Style constants —————

NOISE_ORDER = ["low", "med", "high"]

NOISE_LABEL = ["Low\n( $\sigma = 0.5$  kt)", "Medium\n( $\sigma = 1.0$  kt)", "High\n( $\sigma = 2.0$  kt)"]

PHASE_ORDER = ["climb", "descent"]

PHASE_LABEL = {"climb": "Climb", "descent": "Descent"}


COLOR_T = "#2E86C1" # threshold detector – blue
COLOR_R = "#E67E22" # rate detector – orange
COLOR_FA_T = "#2E86C1"
COLOR_FA_R = "#E67E22"


DPI = 300

FIG_W, FIG_H = 9, 5 # inches – matches thesis figure canvas


FONT_FAMILY = "Times New Roman"

plt.rcParams.update({
    "font.family": FONT_FAMILY,
    "font.size": 12,
    "axes.titlesize": 13,
    "axes.labelsize": 12,
    "xtick.labelsize": 11,
    "ytick.labelsize": 11,
    "legend.fontsize": 11,
    "figure.dpi": DPI,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.4,
    "grid.linestyle": "--"
})

```

```

def load(grouped_path, runs_path):
    if not os.path.exists(grouped_path):
        raise FileNotFoundError(
            f"Grouped summary not found: {grouped_path}\n"
            "Run analyze_results.py first."
        )

    grouped = pd.read_csv(grouped_path)
    runs = pd.read_csv(runs_path)

    print(f"Loaded grouped: {len(grouped)} rows | runs: {len(runs)} rows")

    return grouped, runs


# — Figure 1 – Bias detection delay —————
def fig_bias_delay(grouped, out_dir):
    """
    Grouped bar chart: mean detection delay for bias faults.

    Both detectors side by side, grouped by noise level, separated by phase.
    """

    bias = grouped[grouped["fault_type"] == "bias"].copy()

    bias["noise_idx"] = bias["noise"].map({k: i for i, k in
    enumerate(NOISE_ORDER)})

    bias = bias.sort_values(["phase_focus", "noise_idx"])

    phases = PHASE_ORDER

    x = np.arange(len(NOISE_ORDER))

    width = 0.30

    offset = 0.32

    fig, axes = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), sharey=True)

    fig.suptitle(
        "Figure A – Mean Detection Delay: Bias Faults (Both Detectors)",
        fontsize=13, fontweight="bold", y=1.01
    )

```

```

)

for ax, phase in zip(axes, phases):

    sub = bias[bias["phase_focus"] == phase]

    sub = sub.set_index("noise")

    t_vals = [sub.loc[n, "T_delay_mean"] if n in sub.index else 0
               for n in NOISE_ORDER]

    r_vals = [sub.loc[n, "R_delay_mean"] if n in sub.index else 0
               for n in NOISE_ORDER]

    bars_t = ax.bar(x - width/2, t_vals, width,
                    label="Threshold", color=COLOR_T, alpha=0.85,
                    edgecolor="white", linewidth=0.8)

    bars_r = ax.bar(x + width/2, r_vals, width,
                    label="Rate-of-Change", color=COLOR_R, alpha=0.85,
                    edgecolor="white", linewidth=0.8)

    ax.set_title(f"{PHASE_LABEL[phase]} Phase", fontweight="bold")
    ax.set_xticks(x)
    ax.set_xticklabels(NOISE_LABEL)
    ax.set_xlabel("Noise Level")
    ax.yaxis.set_major_formatter(ticker.FormatStrFormatter("%.2f"))

    # Value labels on bars
    for bar in list(bars_t) + list(bars_r):
        h = bar.get_height()
        if h > 0.001:
            ax.text(bar.get_x() + bar.get_width()/2, h + 0.001,
                    f"{h:.2f}", ha="center", va="bottom", fontsize=9)

axes[0].set_ylabel("Mean Detection Delay (s)")
axes[0].legend(loc="upper left")

```

```

axes[1].legend(loc="upper left")

plt.tight_layout()

path = os.path.join(out_dir, "py_fig_A_bias_delay.png")
fig.savefig(path, dpi=DPI, bbox_inches="tight")

plt.close(fig)

print(f"Saved: {path}")

# — Figure 2 – Freeze missed detection rate —————
def fig_freeze_miss(grouped, out_dir):
    """
    Grouped bar chart: missed detection rate for freeze faults.
    Threshold vs rate-of-change, grouped by noise, split by phase.
    """
    freeze = grouped[grouped["fault_type"] == "freeze"].copy()

    freeze["noise_idx"] = freeze["noise"].map({k: i for i, k in
enumerate(NOISE_ORDER)})

    freeze = freeze.sort_values(["phase_focus", "noise_idx"])

    x = np.arange(len(NOISE_ORDER))
    width = 0.30

    fig, axes = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), sharey=True)
    fig.suptitle(
        "Figure B – Missed Detection Rate: Freeze Faults (Both Detectors)",
        fontsize=13, fontweight="bold", y=1.01
    )

    for ax, phase in zip(axes, PHASE_ORDER):
        sub = freeze[freeze["phase_focus"] == phase].set_index("noise")
        t_vals = [sub.loc[n, "T_miss_rate"] if n in sub.index else 0
                    for n in NOISE_ORDER]

```

```

r_vals = [sub.loc[n, "R_miss_rate"] if n in sub.index else 0
           for n in NOISE_ORDER]

bars_t = ax.bar(x - width/2, t_vals, width,
                label="Threshold", color=COLOR_T, alpha=0.85,
                edgecolor="white")

bars_r = ax.bar(x + width/2, r_vals, width,
                label="Rate-of-Change", color=COLOR_R, alpha=0.85,
                edgecolor="white")

ax.set_title(f"{PHASE_LABEL[phase]} Phase", fontweight="bold")
ax.set_xticks(x)
ax.set_xticklabels(NOISE_LABEL)
ax.set_xlabel("Noise Level")
ax.set_ylim(0, 1.15)
ax.yaxis.set_major_formatter(ticker.PercentFormatter(xmax=1))

for bar in list(bars_t) + list(bars_r):
    h = bar.get_height()
    if h > 0.01:
        ax.text(bar.get_x() + bar.get_width()/2, h + 0.02,
                f"{h:.0%}", ha="center", va="bottom", fontsize=9)

axes[0].set_ylabel("Missed Detection Rate")
axes[0].legend(loc="upper right")
axes[1].legend(loc="upper right")

# Reference line at 100%
for ax in axes:
    ax.axhline(1.0, color="red", linewidth=0.8, linestyle=":",
              alpha=0.6, label="100% miss")

```

```

plt.tight_layout()

path = os.path.join(out_dir, "py_fig_B_freeze_miss.png")

fig.savefig(path, dpi=DPI, bbox_inches="tight")

plt.close(fig)

print(f"Saved: {path}")

# — Figure 3 – False alarm count vs noise —————

def fig_false_alarms(grouped, out_dir):
    """
    Four-panel chart: mean false alarm count per scenario.
    Panels: bias/climb, bias/descent, freeze/climb, freeze/descent.
    """
    fig, axes = plt.subplots(2, 2, figsize=(FIG_W + 2, FIG_H + 2), sharey=False)
    fig.suptitle(
        "Figure C – Mean False Alarm Events per Run",
        fontsize=13, fontweight="bold", y=1.01
    )
    panels = [
        ("bias", "climb", axes[0, 0], "Bias – Climb"),
        ("bias", "descent", axes[0, 1], "Bias – Descent"),
        ("freeze", "climb", axes[1, 0], "Freeze – Climb"),
        ("freeze", "descent", axes[1, 1], "Freeze – Descent"),
    ]

    x = np.arange(len(NOISE_ORDER))
    width = 0.30

    for ft, pf, ax, title in panels:
        sub = grouped[
            (grouped["fault_type"] == ft) &
            (grouped["phase_focus"] == pf)

```

```

].set_index("noise")

t_vals = [sub.loc[n, "T_FA_mean"] if n in sub.index else 0
           for n in NOISE_ORDER]

r_vals = [sub.loc[n, "R_FA_mean"] if n in sub.index else 0
           for n in NOISE_ORDER]

ax.bar(x - width/2, t_vals, width, label="Threshold",
       color=COLOR_T, alpha=0.85, edgecolor="white")

ax.bar(x + width/2, r_vals, width, label="Rate-of-Change",
       color=COLOR_R, alpha=0.85, edgecolor="white")

ax.set_title(title, fontweight="bold", fontsize=11)

ax.set_xticks(x)

ax.set_xticklabels(["Low", "Med", "High"])

ax.set_xlabel("Noise Level")

ax.set_ylabel("Mean False Alarms / Run")

ax.legend(fontsize=9)

plt.tight_layout()

path = os.path.join(out_dir, "py_fig_C_false_alarms.png")

fig.savefig(path, dpi=DPI, bbox_inches="tight")

plt.close(fig)

print(f"Saved: {path}")

# — Figure 4 – Freeze delay distribution (box plots) —————
def fig_freeze_delay_dist(runs, out_dir):
    """
    Box plots: distribution of detection delay for freeze faults,
    rate-of-change detector only (since threshold misses most freeze faults).
    Grouped by noise level, separated by phase.

```

```

"""

freeze = runs[
    (runs["fault_type"] == "freeze") &
    (runs["R_missed"] == 0)
].copy()

fig, axes = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), sharey=True)

fig.suptitle(
    "Figure D – Detection Delay Distribution: Freeze Faults, Rate-of-Change
    Detector",
    fontsize=13, fontweight="bold", y=1.01
)

for ax, phase in zip(axes, PHASE_ORDER):
    sub = freeze[freeze["phase_focus"] == phase]
    data = [sub[sub["noise"] == n]["R_delay_s"].dropna().values
            for n in NOISE_ORDER]

    bp = ax.boxplot(
        data,
        labels=NOISE_LABEL,
        patch_artist=True,
        medianprops=dict(color="black", linewidth=1.5),
        whiskerprops=dict(linewidth=1.2),
        capprops=dict(linewidth=1.2),
        flierprops=dict(marker="o", markersize=3,
                        markerfacecolor=COLOR_R, alpha=0.5)
    )

    for patch in bp["boxes"]:
        patch.set_facecolor(COLOR_R)
        patch.set_alpha(0.6)

    ax.set_title(f"{PHASE_LABEL[phase]} Phase", fontweight="bold")

```

```

        ax.set_xlabel("Noise Level")

    # Annotate sample counts
    for i, d in enumerate(data):
        ax.text(i + 1, ax.get_ylim()[0] if ax.get_ylim()[0] > 0 else 0,
                f"n={len(d)}", ha="center", va="bottom",
                fontsize=9, color="grey")

    axes[0].set_ylabel("Detection Delay (s)")
    plt.tight_layout()
    path = os.path.join(out_dir, "py_fig_D_freeze_delay_dist.png")
    fig.savefig(path, dpi=DPI, bbox_inches="tight")
    plt.close(fig)
    print(f"Saved: {path}")

# — Main —————
def main():
    os.makedirs(FIG_DIR, exist_ok=True)

    grouped, runs = load(GROUPED_PATH, RUNS_PATH)

    print("\nGenerating Figure A – Bias detection delay...")
    fig_bias_delay(grouped, FIG_DIR)

    print("Generating Figure B – Freeze missed detection rate...")
    fig_freeze_miss(grouped, FIG_DIR)

    print("Generating Figure C – False alarm counts...")
    fig_false_alarms(grouped, FIG_DIR)

    print("Generating Figure D – Freeze delay distributions...")

```

```
fig_freeze_delay_dist(runs, FIG_DIR)

print(f"\nAll figures saved to: {FIG_DIR}")
print("Plot generation complete.")

if __name__ == "__main__":
    main()
```

APPENDIX NO. 12

PYTHON ENVIRONMENT AND EXECUTION INSTRUCTIONS

Requirements

Python 3.8 or later is required. The following third-party packages must be installed: pandas (version 1.3 or later), numpy (version 1.21 or later), and matplotlib (version 3.4 or later).

Installation

Install all dependencies using pip by running the following command in a terminal:

```
pip install pandas numpy matplotlib
```

Alternatively, if using Anaconda:

```
conda install pandas numpy matplotlib
```

Execution Order

The scripts must be executed in the following order. First, run `main_run_batch.m` in MATLAB to generate the simulation results (`summary_runs.csv`). Second, run `analyze_results.py` to compute grouped statistics and produce the thesis tables. Third, run `plot_results.py` to generate the result figures.

Working Directory

Both Python scripts must be executed from the root project directory — the same directory that contains the `output/` folder. The scripts use relative paths to locate input files and write output files.

Command Line Execution

Navigate to the project root directory in a terminal and run:

```
python analyze_results.py
python plot_results.py
```

On systems where both Python 2 and Python 3 are installed, use `python3` instead of `python`.

Output Files

`analyze_results.py` writes to `output/summaries/`. `plot_results.py` writes four PNG figures to `output/figures/`. All output directories are created automatically if they do not already exist.